

Data Structure Problems

04

In This Edition

1	Contents	4
2	Arrays	4
	2.1 Two Sum (LC 1)	4
	2.2 Best Time to Buy and Sell Stock (LC 121)	5
	2.3 Maximum Subarray / Kadane's (LC 53)	6
	2.4 Container With Most Water (LC 11)	6
	2.5 Trapping Rain Water (LC 42)	7
	2.6 Rotate Array (LC 189)	8
	2.7 Product of Array Except Self (LC 238)	9
	2.8 Merge Intervals (LC 56)	9
3	Strings	10
	3.1 Longest Substring Without Repeating Characters (LC 3)	10
	3.2 Minimum Window Substring (LC 76)	11
	3.3 Group Anagrams (LC 49)	12
	3.4 Valid Palindrome (LC 125)	12
	3.5 Encode and Decode Strings (LC 271)	13
	3.6 Longest Palindromic Substring (LC 5)	14
	3.7 String to Integer (atoi) (LC 8)	15
4	Hash Maps & Sets	16
	4.1 Two Sum (LC 1)	16
	4.2 Top K Frequent Elements (LC 347)	16
	4.3 Longest Consecutive Sequence (LC 128)	17
	4.4 Subarray Sum Equals K (LC 560)	17
	4.5 Valid Sudoku (LC 36)	18
	4.6 Find All Anagrams in a String (LC 438)	19
5	Linked Lists	20
	5.1 Reverse Linked List (LC 206)	20
	5.2 Merge Two Sorted Lists (LC 21)	21
	5.3 Linked List Cycle (LC 141)	21
	5.4 Reorder List (LC 143)	22
	5.5 Remove Nth Node From End (LC 19)	23
	5.6 Merge K Sorted Lists (LC 23)	24
6	Stacks & Queues	24
	6.1 Valid Parentheses (LC 20)	25
	6.2 Min Stack (LC 155)	25
	6.3 Daily Temperatures (LC 739)	26
	6.4 Largest Rectangle in Histogram (LC 84)	27
	6.5 Sliding Window Maximum (LC 239)	27
	6.6 Evaluate Reverse Polish Notation (LC 150)	28

7.3	Same Tree (LC 100)	30
7.4	Binary Tree Level Order Traversal (LC 102)	31
7.5	Validate Binary Search Tree (LC 98)	32
7.6	Lowest Common Ancestor of a BST (LC 235)	33
7.7	Kth Smallest Element in a BST (LC 230)	33
7.8	Serialize and Deserialize Binary Tree (LC 297)	34
7.9	Diameter of Binary Tree (LC 543)	35
8	Heaps / Priority Queues	36
8.1	Kth Largest Element in an Array (LC 215)	36
8.2	K Closest Points to Origin (LC 973)	37
8.3	Find Median from Data Stream (LC 295)	38
8.4	Task Scheduler (LC 621)	39
8.5	Merge K Sorted Lists (LC 23)	39
9	Tries (Prefix Trees)	39
9.1	Implement Trie (LC 208)	40
9.2	Word Search II (LC 212)	41
9.3	Replace Words (LC 648)	42
9.4	Design Search Autocomplete System (LC 642)	43
10	Graphs	44
10.1	Number of Islands (LC 200)	44
10.2	Clone Graph (LC 133)	45
10.3	Pacific Atlantic Water Flow (LC 417)	46
10.4	Course Schedule (LC 207)	47
10.5	Number of Connected Components in an Undirected Graph (LC 323)	48
10.6	Word Ladder (LC 127)	49
10.7	Network Delay Time (LC 743)	50
10.8	Cheapest Flights Within K Stops (LC 787)	50
11	Union-Find / Disjoint Set	51
11.1	Number of Provinces (LC 547)	52
11.2	Redundant Connection (LC 684)	53
11.3	Accounts Merge (LC 721)	53
11.4	Satisfiability of Equality Equations (LC 990)	54
12	LRU / LFU Cache	55
12.1	LRU Cache (LC 146)	55
12.2	LFU Cache (LC 460)	57
13	Segment Tree & Fenwick Tree (BIT)	58

13.1 Range Sum Query --- Mutable (LC 307)	58
13.2 Count of Smaller Numbers After Self (LC 315)	59
13.3 Reverse Pairs (LC 493)	60

Companion to **Data Structures Deep Dive**. Every "Common Interview Question" listed in that file is solved here in full: the genuinely optimal algorithm, the key idea that makes it work, clean runnable Python, and the specific bugs that trip candidates up. Problems are grouped by the data structure they exercise.

For the structures themselves (mechanics, complexities, when to reach for each), see [02-data-structures.md](#). For reusable pattern templates, see [01-patterns-and-templates.md](#). The pure-algorithm problems (sorting, binary search, graph algorithms, DP, backtracking, greedy) live in [05-algorithm-problems.md](#).

How to read each entry: *Problem* → *Optimal approach & complexity* → *Idea (the insight)* → *Code* → *Error-prone spots* → *Follow-up*.

1 Contents

- › Arrays
- › Strings
- › Hash Maps & Sets
- › Linked Lists
- › Stacks & Queues
- › Trees
- › Heaps / Priority Queues
- › Tries (Prefix Trees)
- › Graphs
- › Union-Find / Disjoint Set
- › LRU / LFU Cache
- › Segment Tree & Fenwick Tree (BIT)

2 Arrays

The array patterns below revolve around scanning once while maintaining running state (a hash map, a best-so-far value, or two converging pointers), trading extra space or clever index math for a linear pass that beats the naive $O(n^2)$.

2.1 Two Sum (LC 1)

Problem. Given an integer array `nums` and an integer `target`, return the indices $i \neq j$ such that `nums[i] + nums[j] = target`. Exactly one solution exists; you may not reuse the same element. Values may be negative; duplicates are allowed (e.g. `[3,3]`, `target=6`).

Optimal — one-pass hash map · Time $O(n)$, Space $O(n)$. Beats the $O(n^2)$ brute force of checking every pair.

Idea. As you scan, for each value x you need a previously seen `target - x`. A hash map from value to index lets you answer "have I seen the complement?" in $O(1)$. Checking the map *before* inserting the current element guarantees $i \neq j$ and handles duplicate values correctly.

```

1 from typing import List
2
3 def two_sum(nums: List[int], target: int) → List[int]:
4     seen = {} # value → index
5     for i, x in enumerate(nums):
6         need = target - x
7         if need in seen: # complement was seen earlier ⇒ i ≠ j
8             return [seen[need], i]
9         seen[x] = i # insert AFTER the lookup
10    return [] # unreachable per problem guarantee

```

**** Error-prone spots:****

- › Inserting `nums[i]` into the map *before* the lookup — a value equal to `target/2` would then match itself, returning `[i, i]`.
- › Storing index→value instead of value→index, forcing an $O(n)$ reverse search.
- › Overwriting a duplicate's index can be fine here (we return on first match), but if you needed the *first* occurrence you must guard with `if x not in seen`.
- › Returning the values instead of the indices.
- › Assuming sorted input and reaching for two pointers — that costs $O(n \log n)$ to sort and loses original indices.

Follow-up. If the array is already sorted (LC 167), use two pointers from both ends for $O(n)$ time and $O(1)$ space.

2.2 Best Time to Buy and Sell Stock (LC 121)

Problem. `prices[i]` is the stock price on day `i`. Choose one buy day and a strictly later sell day to maximize profit; return the max profit, or 0 if no positive profit is possible. Single transaction only.

Optimal — running minimum · Time $O(n)$, Space $O(1)$. Beats the $O(n^2)$ all-pairs comparison.

Idea. The best sell on day `i` pairs with the cheapest price seen on any earlier day. Track `min_price` so far and, at each day, the profit if you sold today is `price - min_price`. Keep the maximum such profit. The buy-before-sell constraint is satisfied because `min_price` only reflects prior days.

```

1 from typing import List
2
3 def max_profit(prices: List[int]) → int:
4     min_price = float('inf')
5     best = 0
6     for p in prices:
7         min_price = min(min_price, p) # cheapest buy up to today
8         best = max(best, p - min_price) # best sale if selling today
9     return best

```

**** Error-prone spots:****

- › Updating `best` before `min_price` on the same iteration can let you “buy and sell” on the same day giving a spurious profit; updating `min_price` first keeps `p - min_price ≥ 0` consistent (it yields 0, which is harmless).

- › Initializing best to $-\infty$ instead of 0, returning a loss when prices only fall.
- › Empty array: the loop runs zero times and returns 0, which is correct — but `min(prices)` based solutions crash.
- › Confusing this with "buy/sell multiple times" (LC 122), which greedily sums every positive delta.

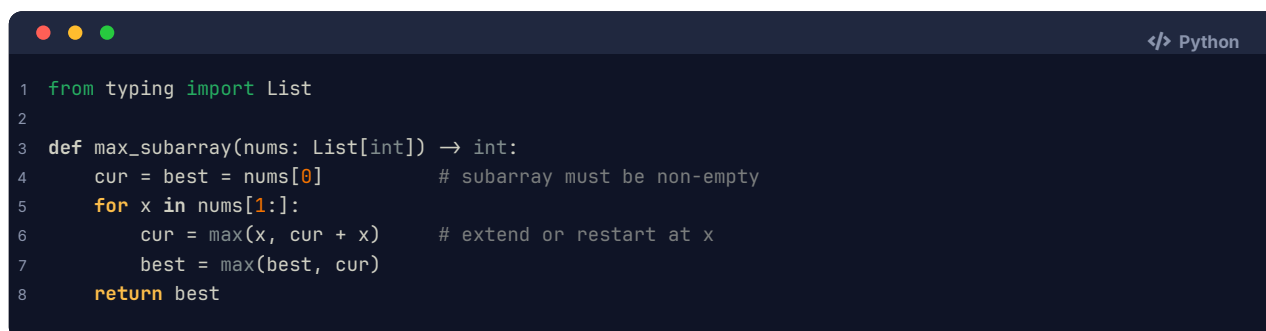
Follow-up. LC 122 (unlimited transactions): $\sum \max(0, \text{prices}[i] - \text{prices}[i-1])$ over all i .

2.3 Maximum Subarray / Kadane's (LC 53)

Problem. Find the contiguous, non-empty subarray with the largest sum and return that sum. Array can be all negative (answer is the largest single element). Length ≥ 1 .

Optimal — Kadane's DP · Time $O(n)$, Space $O(1)$. Beats $O(n^2)$ prefix-pair and $O(n \log n)$ divide-and-conquer.

Idea. Let `cur` be the max subarray sum ending exactly at the current index. Either extend the previous best (`cur + x`) or start fresh at `x`, whichever is larger: `cur = max(x, cur + x)`. The global answer is the max of all `cur`. Starting fresh handles the case where prior sum is negative drag.



```

1 from typing import List
2
3 def max_subarray(nums: List[int]) → int:
4     cur = best = nums[0]      # subarray must be non-empty
5     for x in nums[1:]:
6         cur = max(x, cur + x)  # extend or restart at x
7         best = max(best, cur)
8     return best

```

**** Error-prone spots:****

- › Initializing `best = 0` breaks all-negative inputs (would wrongly return 0); seed both from `nums[0]`.
- › Empty array: `nums[0]` raises `IndexError` — clarify the non-empty constraint or guard explicitly.
- › Using `cur = max(0, cur + x)` (the "reset to 0" variant) silently assumes a non-negative answer is allowed.
- › Forgetting to update `best` after recomputing `cur`, or updating it before.

Follow-up. To also return the indices, track a start pointer that resets whenever you restart (`cur = x`).

2.4 Container With Most Water (LC 11)

Problem. Given heights `height[i]`, pick two lines $i < j$ so the water area $\min(\text{height}[i], \text{height}[j]) * (j - i)$ is maximized. Return that area. Length ≥ 2 .

Optimal — two pointers · Time $O(n)$, Space $O(1)$. Beats $O(n^2)$ all pairs.

Idea. Start with the widest container (pointers at both ends). The area is bounded by the *shorter* line, so moving the taller line inward can only shrink width without lifting the limiting height — it can never help. Moving the shorter line is the only move that might find a taller bound, so always advance the shorter pointer.

```

1 from typing import List
2
3 def max_area(height: List[int]) → int:
4     i, j = 0, len(height) - 1
5     best = 0
6     while i < j:
7         area = min(height[i], height[j]) * (j - i)
8         best = max(best, area)
9         if height[i] < height[j]: # move the shorter wall inward
10            i += 1
11        else:
12            j -= 1
13    return best

```

**** Error-prone spots:****

- › Moving the taller pointer (or always moving i) — provably wrong, you can skip the optimum.
- › Width is $j - i$, not $j - i + 1$ (this measures span between lines, not inclusive cells).
- › On ties $\text{height}[i] = \text{height}[j]$, either pointer may move; just be consistent.
- › Confusing this with Trapping Rain Water — here only two walls hold water, not the whole skyline.

2.5 Trapping Rain Water (LC 42)

Problem. Given an elevation map $\text{height}[i]$ (bar widths 1), compute total trapped rainwater after rain. Heights are non-negative; array may be empty.

Optimal — two pointers, $O(1)$ space · Time $O(n)$, Space $O(1)$. Beats the $O(n)$ time / $O(n)$ space prefix-max-arrays approach and the $O(n^2)$ per-cell scan.

Idea. Water above cell i is $\min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$. Maintain left_max and right_max from both ends. The key invariant: if $\text{left_max} \leq \text{right_max}$, then for the left pointer the binding constraint is left_max (some wall at least $\text{right_max} \geq \text{left_max}$ exists on the right), so water at left is fully determined by left_max — process and advance left . Symmetrically otherwise.

```

1 from typing import List
2
3 def trap(height: List[int]) → int:
4     if not height:
5         return 0
6     i, j = 0, len(height) - 1
7     left_max, right_max = height[i], height[j]
8     water = 0
9     while i < j:
10        if left_max ≤ right_max:
11            i += 1
12            left_max = max(left_max, height[i])
13            water += left_max - height[i] # ≥ 0 by construction
14        else:
15            j -= 1
16            right_max = max(right_max, height[j])
17            water += right_max - height[j]
18    return water

```

**** Error-prone spots:****

- › Updating `left_max/right_max` *before* moving the pointer adds 0 for the current cell or double counts; advance first, then update max, then add.
- › Comparing `height[i]` vs `height[j]` instead of `left_max` vs `right_max` — that breaks the invariant.
- › Empty input: must early-return before indexing `height[-1]`.
- › Adding negative water when the current bar is the tallest so far — the $\leq$ /update ordering keeps the term non-negative.

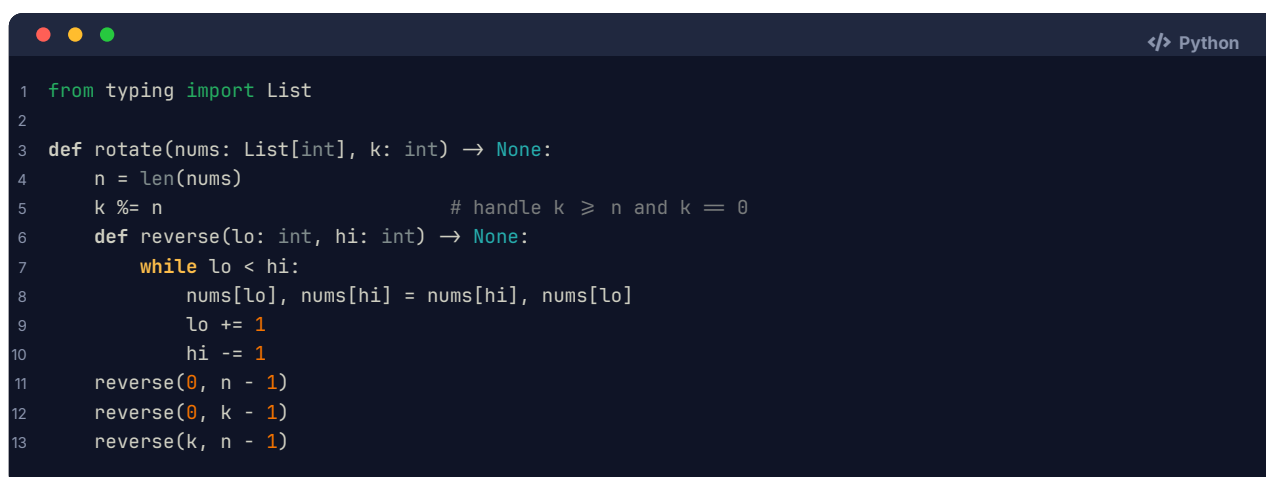
Follow-up. A monotonic decreasing stack also solves it in $O(n)$ by filling water layer-by-layer between popped bars.

2.6 Rotate Array (LC 189)

Problem. Rotate `nums` to the right by `k` steps, in place. `k` may exceed `n`; `k` can be 0. Modify the array, return nothing.

Optimal — reverse three times · Time $O(n)$, Space $O(1)$. Beats the $O(n)$ extra-array copy and the $O(n*k)$ one-step-at-a-time shift.

Idea. Right-rotating by `k` moves the last `k` elements to the front. Reverse the whole array, then reverse the first `k`, then reverse the remaining `n - k`. Each reversal flips order; the composition lands every element in its rotated position. Reduce `k` modulo `n` first since rotating by `n` is a no-op.



```

1 from typing import List
2
3 def rotate(nums: List[int], k: int) -> None:
4     n = len(nums)
5     k %= n          # handle k ≥ n and k = 0
6     def reverse(lo: int, hi: int) -> None:
7         while lo < hi:
8             nums[lo], nums[hi] = nums[hi], nums[lo]
9             lo += 1
10            hi -= 1
11    reverse(0, n - 1)
12    reverse(0, k - 1)
13    reverse(k, n - 1)

```

**** Error-prone spots:****

- › Forgetting `k %= n`; with `k ≥ n` the sub-reversals get wrong bounds. Also guards against division/reverse(0, -1) issues when `k = 0` (the while loop simply does nothing).
- › `n = 0` would make `k %= n` divide by zero — guard if empty arrays are allowed.
- › Reversing segments `[0, k]` and `[k+1, n-1]` (wrong split) instead of `[0, k-1]` and `[k, n-1]`.
- › Left rotation by `k` is right rotation by `n - k`; don't mix the direction.

Follow-up. The cyclic-replacement (juggling) algorithm also achieves $O(n)/O(1)$ using $\text{gcd}(n, k)$ cycles, but is far easier to get wrong.

2.7 Product of Array Except Self (LC 238)

Problem. Return out where out[i] is the product of all elements except nums[i]. Must run in $O(n)$ **without division** (the array may contain zeros). The output array does not count as extra space.

Optimal — prefix/suffix products · Time $O(n)$, Space $O(1)$ extra. Beats the $O(n^2)$ recompute and the division trick (which fails on zeros).

Idea. out[i] = (product of everything left of i) * (product of everything right of i). First pass writes left-products into out. Second pass walks right-to-left maintaining a running right product and multiplies it in. No division, and zeros are handled naturally.

```

1 from typing import List
2
3 def product_except_self(nums: List[int]) → List[int]:
4     n = len(nums)
5     out = [1] * n
6     for i in range(1, n):          # out[i] = product of nums[0..i-1]
7         out[i] = out[i - 1] * nums[i - 1]
8     right = 1
9     for i in range(n - 1, -1, -1): # fold in product of nums[i+1..n-1]
10        out[i] *= right
11        right *= nums[i]
12    return out

```

**** Error-prone spots:****

- Seeding out[0] or right to 0 instead of 1 zeroes out everything.
- Updating right *= nums[i] *before* out[i] *= right, which wrongly includes nums[i] itself.
- Off-by-one in the prefix pass: out[i] must use nums[i-1], not nums[i].
- Reaching for division and special-casing zeros — fragile when there are two or more zeros.

Follow-up. If division were permitted: handle 0 zeros (divide directly), 1 zero (only that slot is the product of the rest, others are 0), 2+ zeros (all zero).

2.8 Merge Intervals (LC 56)

Problem. Given intervals [start, end], merge all overlapping intervals and return the non-overlapping set. Treat touching intervals ([1, 4], [4, 5]) as overlapping. Input may be unsorted; output order by start.

Optimal — sort then sweep · Time $O(n \log n)$, Space $O(n)$ output ($O(1)$ extra besides sort). The sort dominates; you cannot do better than $O(n \log n)$ in the comparison model.

Idea. Sort by start. Walk through; keep the last interval in the result. If the current interval starts at or before the last one's end, they overlap — extend the last end to max(last_end, cur_end). Otherwise it is disjoint, so append it as a new block.

```

1 from typing import List
2
3 def merge(intervals: List[List[int]]) → List[List[int]]:
4     intervals.sort(key=lambda iv: iv[0])

```

```

5     merged = []
6     for start, end in intervals:
7         if merged and start ≤ merged[-1][1]: # overlaps/touches previous
8             merged[-1][1] = max(merged[-1][1], end)
9         else:
10            merged.append([start, end])
11     return merged

```

**** Error-prone spots:****

- › Using `start < merged[-1][1]` (strict) drops the touching case `[1,4], [4,5]` that should merge.
- › Setting `merged[-1][1] = end` instead of `max(...)` shrinks the end when the current interval is nested inside the previous `([1,10], [2,3])`.
- › Forgetting to sort, or sorting by end instead of start.
- › Mutating shared sublist references — appending `[start, end]` (a fresh list) avoids aliasing the input rows.

Follow-up. Insert Interval (LC 57): with an already-sorted list you can do a single $O(n)$ pass without re-sorting.

3 Strings

String problems here lean on sliding windows over character-count state, canonical-form hashing for grouping, and careful pointer arithmetic — almost always reducible to a single linear scan.

3.1 Longest Substring Without Repeating Characters (LC 3)

Problem. Return the length of the longest substring of `s` with all distinct characters. `s` may be empty; characters can be any Unicode. Substring must be contiguous.

Optimal — sliding window with last-seen map · Time $O(n)$, Space $O(\min(n, \text{alphabet}))$. Beats $O(n^2)$ substring checks.

Idea. Maintain a window `[left, right]` with no repeats. Store each character's last index. When the right char was seen at index $\geq \text{left}$, jump `left` to one past that prior occurrence (never move `left` backward). The window stays repeat-free, and its max length is the answer.

```

1 def length_of_longest_substring(s: str) → int:
2     last = {} # char → most recent index
3     left = 0
4     best = 0
5     for right, c in enumerate(s):
6         if c in last and last[c] ≥ left:
7             left = last[c] + 1 # shrink window past the duplicate
8         last[c] = right
9         best = max(best, right - left + 1)
10    return best

```

**** Error-prone spots:****

- › Omitting the `last[c] ≥ left` guard lets a stale occurrence outside the window move `left` backward, corrupting the length.
- › Setting `left = last[c]` instead of `last[c] + 1` keeps the duplicate inside the window.
- › Window length is `right - left + 1` (inclusive); forgetting the `+ 1` undercounts.
- › Updating `last[c]` before computing the new `left` reads/writes in the wrong order.

Follow-up. "At most K distinct characters" (LC 340) uses the same window but shrinks while the count map has more than K keys.

3.2 Minimum Window Substring (LC 76)

Problem. Given `s` and `t`, return the smallest substring of `s` containing every character of `t` including multiplicities; return `""` if none exists. `t` may have duplicates; `s/t` may be empty.

Optimal – sliding window with need counts · Time $O(|s| + |t|)$, Space $O(|t|)$. Beats the $O(|s|^2)$ check-every-window approach.

Idea. Count required characters in `need`. Expand `right`, decrementing `missing`; track `missing = total required chars still unmet`. When `missing = 0`, the window is valid – contract `left` as far as possible while it stays valid, recording the smallest. A character only re-adds to `missing` when its count in `need` goes strictly positive, so surplus characters never block contraction.

```

1  from collections import Counter
2
3  def min_window(s: str, t: str) → str:
4      if not t or not s:
5          return ""
6      need = Counter(t)
7      missing = len(t)          # counts duplicates in t
8      left = 0
9      best_len = float('inf')
10     best_l = 0
11     for right, c in enumerate(s):
12         if need[c] > 0:        # this char helps satisfy t
13             missing -= 1
14         need[c] -= 1           # may go negative for surplus chars
15         while missing == 0:    # window valid: try to shrink
16             if right - left + 1 < best_len:
17                 best_len = right - left + 1
18                 best_l = left
19             need[s[left]] += 1
20             if need[s[left]] > 0: # removing this char breaks validity
21                 missing += 1
22             left += 1
23     return "" if best_len == float('inf') else s[best_l:best_l + best_len]

```

**** Error-prone spots:****

- › Decrementing `missing` for *every* char rather than only when `need[c] > 0` – surplus/irrelevant chars must not reduce `missing`.
- › The symmetric bug on contraction: only bump `missing` back up when `need[s[left]]` becomes strictly positive after the increment.
- › Returning the length instead of the substring, or slicing with stale `best_l`.
- › Using `need[c] ≥ 0` anywhere instead of `> 0`; the sign distinguishes "needed" from "surplus".
- › Empty `t` or `s` not guarded.

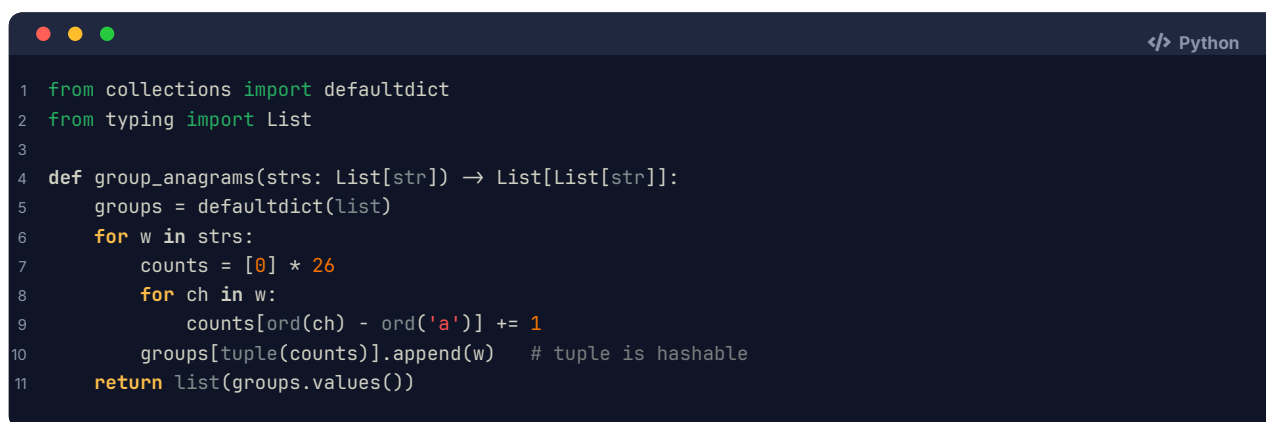
Follow-up. "Minimum window with all distinct chars of t" or fixed-K variants reuse the same expand/contract skeleton.

3.3 Group Anagrams (LC 49)

Problem. Given a list of strings, group those that are anagrams of one another. Return a list of groups (order within and across groups is unconstrained). Strings are lowercase letters; empty strings group together.

Optimal — canonical key via char-count tuple · Time $O(n * L)$, Space $O(n * L)$ where L is max word length and the alphabet is fixed (26). Sorting keys gives $O(n * L \log L)$; the count-tuple key removes the $\log L$.

Idea. Two strings are anagrams iff they share a canonical form. A length-26 count tuple of letter frequencies is such a canonical, hashable key and is built in $O(L)$ without sorting. Bucket strings into a dict keyed by this tuple.



```

1 from collections import defaultdict
2 from typing import List
3
4 def group_anagrams(strs: List[str]) -> List[List[str]]:
5     groups = defaultdict(list)
6     for w in strs:
7         counts = [0] * 26
8         for ch in w:
9             counts[ord(ch) - ord('a')] += 1
10        groups[tuple(counts)].append(w) # tuple is hashable
11    return list(groups.values())

```

**** Error-prone spots:****

- › Using a list (unhashable) as the dict key instead of a tuple.
- › The count-array key only works for a fixed alphabet; for arbitrary Unicode fall back to `sorted(w)` joined into a string.
- › Forgetting that empty strings are valid and form their own group (key = all zeros).
- › Mutating and reusing one counts array across words without resetting it.

Follow-up. For Unicode or case-sensitive input, key on `' '.join(sorted(w))` — $O(n * L \log L)$ but fully general.

3.4 Valid Palindrome (LC 125)

Problem. Return True if `s` is a palindrome considering only alphanumeric characters and ignoring case. Empty string (or all non-alphanumeric) counts as a palindrome.

Optimal — two pointers · Time $O(n)$, Space $O(1)$. Beats building a filtered, lowercased copy ($O(n)$ extra space).

Idea. Two pointers from both ends. Skip non-alphanumeric characters on each side, then compare the lowercased characters. Mismatch means not a palindrome; pointers meeting means success.

```

1 def is_palindrome(s: str) → bool:
2     i, j = 0, len(s) - 1
3     while i < j:
4         if not s[i].isalnum():
5             i += 1
6         elif not s[j].isalnum():
7             j -= 1
8         else:
9             if s[i].lower() ≠ s[j].lower():
10                return False
11            i += 1
12            j -= 1
13    return True

```

**** Error-prone spots:****

- › Advancing both pointers in the same iteration while one still points at punctuation — skip one side at a time.
- › Forgetting `.lower()` (or normalizing only one side) breaks mixed-case palindromes.
- › `isalnum()` in Python also accepts Unicode letters/digits; restrict to ASCII (`c.isascii()` and `c.isalnum()`) if the spec requires it.
- › Infinite loop if you skip without ever advancing on a matched comparison.

Follow-up. LC 680 (allow one deletion): on the first mismatch, check whether skipping either `s[i]` or `s[j]` yields a palindrome.

3.5 Encode and Decode Strings (LC 271)

Problem. Design `encode(list[str]) → str` and `decode(str) → list[str]` so any list of strings round-trips. Strings may contain any character including digits, delimiters, and `#`; the list may be empty or contain empty strings.

Optimal — length-prefix protocol · Time $O(\text{total chars})$, Space $O(\text{total chars})$. A naive delimiter join fails because any chosen delimiter can appear in the data.

Idea. Prefix each string with its length and a sentinel: `"<len>#<string>"`. On decode, read digits up to the `#` to get the length, then take exactly that many characters as the payload — content is never scanned for delimiters, so any character (including `#`) is safe.

```

1 from typing import List
2
3 def encode(strs: List[str]) → str:
4     return ''.join(f"{len(s)}#{s}" for s in strs)
5
6 def decode(data: str) → List[str]:
7     res = []
8     i = 0
9     n = len(data)
10    while i < n:
11        j = i
12        while data[j] ≠ '#':      # read the length prefix
13            j += 1
14        length = int(data[i:j])
15        start = j + 1             # payload begins after '#'

```

```

16     res.append(data[start:start + length])
17     i = start + length          # jump past payload to next record
18     return res

```

**** Error-prone spots:****

- › Using a plain separator (e.g. comma or space) — breaks the moment that character appears in a string.
- › Off-by-one around #: payload starts at $j + 1$, and the next record starts at $start + length$.
- › Not handling length 0 (empty string) — the slice `data[start:start+0]` correctly yields "", but only if indices are right.
- › Searching for # in the payload instead of trusting the length count.
- › Empty list encodes to "" and must decode back to [] (the while loop simply never runs).

Follow-up. Use a fixed-width (e.g. 4-byte big-endian) length header to avoid scanning for # at all, useful for binary protocols.

3.6 Longest Palindromic Substring (LC 5)

Problem. Return the longest contiguous substring of s that is a palindrome (any one if ties). s length ≥ 1 ; single characters are palindromes.

Optimal — expand around center · Time $O(n^2)$, Space $O(1)$. Beats the $O(n^2)$ time / $O(n^2)$ space DP. Manacher's algorithm reaches $O(n)$ (see follow-up).

Idea. Every palindrome has a center: either a character (odd length) or a gap between two characters (even length). There are $2n - 1$ centers. From each, expand outward while characters match; track the longest span found. $O(1)$ extra space since we only keep indices.

```

1 def longest_palindrome(s: str) → str:
2     if not s:
3         return ""
4     start, end = 0, 0          # best [start, end] inclusive
5
6     def expand(l: int, r: int) → None:
7         nonlocal start, end
8         while l ≥ 0 and r < len(s) and s[l] == s[r]:
9             l -= 1
10            r += 1
11            # loop overshoots by one on each side
12            if r - l - 2 > end - start:
13                start, end = l + 1, r - 1
14
15    for i in range(len(s)):
16        expand(i, i)           # odd-length center
17        expand(i, i + 1)       # even-length center
18    return s[start:end + 1]

```

**** Error-prone spots:****

- › After the expand loop, the valid palindrome is `s[l+1 .. r-1]` because the loop exits one step past the match; using `l, r` directly is wrong.
- › Forgetting the even-length centers (`expand(i, i+1)`) misses palindromes like "abba".

- › Length comparison $r - l - 2$ vs $end - start$ (current best length) — off-by-one here picks the wrong substring.
- › Returning `s[start:end]` instead of `s[start:end + 1]` (end is inclusive).

Follow-up. Manacher's algorithm computes all palindrome radii in $O(n)$ by mirroring already-known centers; worth knowing when $O(n^2)$ is too slow.

3.7 String to Integer (atoi) (LC 8)

Problem. Parse a leading optional-signed integer from `s`: skip leading spaces, read an optional `+/-`, then consecutive digits, stopping at the first non-digit. Ignore trailing junk. Clamp the result to the 32-bit signed range $[-2^{31}, 2^{31} - 1]$. Return 0 if no digits parse.

Optimal — single linear scan · Time $O(n)$, Space $O(1)$. No faster approach exists; the work is inherent.

Idea. Process in strict phases: (1) skip spaces, (2) read at most one sign, (3) accumulate digits as `result = result * 10 + digit`, clamping at each step to avoid overflow, (4) stop at the first non-digit. Clamp eagerly so even arbitrarily long digit runs cannot overflow.



```

1 def my_atoi(s: str) → int:
2     INT_MIN, INT_MAX = -2**31, 2**31 - 1
3     i, n = 0, len(s)
4     while i < n and s[i] == ' ':          # 1) skip leading spaces only
5         i += 1
6     sign = 1
7     if i < n and s[i] in '+-':           # 2) at most one sign
8         sign = -1 if s[i] == '-' else 1
9         i += 1
10    result = 0
11    while i < n and s[i].isdigit():      # 3) accumulate digits
12        result = result * 10 + int(s[i])
13        if sign == 1 and result > INT_MAX:
14            return INT_MAX
15        if sign == -1 and -result < INT_MIN:
16            return INT_MIN
17        i += 1
18    return sign * result                  # 4) implicit stop at non-digit

```

**** Error-prone spots:****

- › Skipping all whitespace (tabs/newlines) — the spec skips only the space character ' '.
- › Allowing a sign after digits, or multiple signs ("`+ -2`" should parse as 0).
- › Clamping only at the end: with Python's big ints it still works, but in fixed-width languages you must clamp *during* accumulation; do it here for portability and to short-circuit huge inputs.
- › `s[i].isdigit()` accepts some non-ASCII digits in Python; use `s[i] in '0123456789'` if strict ASCII is required.
- › Returning the wrong clamp bound for the sign (`INT_MIN` vs `INT_MAX`).

Follow-up. A strict-parser variant rejects any trailing non-digit characters instead of ignoring them — a common interview twist.

4 Hash Maps & Sets

Hash maps and sets give expected $O(1)$ insert, delete, and membership tests, turning many nested-loop scans into single passes by trading space for time. The recurring tricks are: cache what you've seen, count frequencies, and store prefix aggregates keyed by value.

4.1 Two Sum (LC 1)

See the Arrays section — the canonical hash-map lookup problem.

4.2 Top K Frequent Elements (LC 347)

Problem. Given an integer array `nums` and an integer `k`, return the `k` most frequent elements, in any order. It is guaranteed the answer is unique. $1 \leq k \leq \text{number of distinct values}$.

Optimal — count + bucket sort by frequency · Time $O(n)$, Space $O(n)$. Beats the naive "sort by frequency" $O(n \log n)$ and the heap-based $O(n \log k)$.

Idea. A value's frequency is at most n , so create $n + 1$ buckets indexed by frequency and drop each distinct value into the bucket equal to its count. Walking buckets from high frequency to low and collecting until we have `k` elements yields the answer in linear time because the total work across all buckets is the number of distinct values. The heap variant keeps a size-`k` min-heap of $(\text{count}, \text{value})$ and is preferable only when `k` is tiny relative to `n` or you want a streaming solution.

```
1 from collections import Counter
2
3 def top_k_frequent(nums: list[int], k: int) -> list[int]:
4     count = Counter(nums)
5     # buckets[f] holds all values that appear exactly f times; max f is len(nums)
6     buckets = [[] for _ in range(len(nums) + 1)]
7     for val, freq in count.items():
8         buckets[freq].append(val)
9
10    result = []
11    for freq in range(len(buckets) - 1, 0, -1): # high freq -> low
12        for val in buckets[freq]:
13            result.append(val)
14            if len(result) == k:
15                return result
16    return result
```

**** Error-prone spots:****

- Allocating only n buckets instead of $n + 1$: a value can appear n times, so index n must exist.
- Iterating buckets low-to-high and returning the *least* frequent by mistake — sweep from the top index down.
- Forgetting to skip bucket index 0 (no value has frequency 0); harmless but include the lower bound 0 exclusive to avoid confusion.
- Returning frequencies instead of the values themselves.
- Off-by-one in the early-return check: stop the moment `len(result) == k`, not after the inner loop completes.

Follow-up. With a `heapq.nlargest(k, count.keys(), key=count.get)` you get a clean $O(n \log k)$ one-liner — know it as the “bounded memory” alternative.

4.3 Longest Consecutive Sequence (LC 128)

Problem. Given an unsorted array `nums`, return the length of the longest run of consecutive integers (e.g. `[100, 4, 200, 1, 3, 2]` -> 4 for 1, 2, 3, 4). Duplicates are allowed but don't extend a run. Must run in $O(n)$.

Optimal — hash set + start-of-run check · Time $O(n)$, Space $O(n)$. Beats the obvious sort-then-scan $O(n \log n)$.

Idea. Put every value in a set for $O(1)$ membership. A number x is the *start* of a run only if $x - 1$ is absent; from each such start, walk $x+1, x+2, \dots$ while they're present and measure the run. Each element is visited by an inner walk at most once (only from its run's start), so the total work is $O(n)$ despite the nested look.

```

1 def longest_consecutive(nums: List[int]) -> int:
2     num_set = set(nums)
3     best = 0
4     for x in num_set:
5         if x - 1 not in num_set:           # only expand from a run's start
6             length = 1
7             while x + length in num_set:
8                 length += 1
9             best = max(best, length)
10    return best

```

**** Error-prone spots:****

- › Iterating over `nums` instead of `num_set`: duplicates cause repeated inner walks and can blow up to $O(n^2)$ on inputs like all-equal values.
- › Omitting the `x - 1 not in num_set` guard — without it the algorithm is quadratic, not linear.
- › Returning 0 vs handling the empty array: `best = 0` already covers `nums == []`.
- › Using a list for membership tests instead of a set (turns $O(1)$ into $O(n)$).
- › Building consecutive checks with `x + 1` only once instead of looping with a growing offset.

4.4 Subarray Sum Equals K (LC 560)

Problem. Given an integer array `nums` (may contain negatives and zeros) and integer `k`, return the number of contiguous subarrays whose sum equals `k`. Count overlapping subarrays separately.

Optimal — prefix sums + hashmap of counts · Time $O(n)$, Space $O(n)$. Beats the $O(n^2)$ all-pairs prefix scan; a sliding window does NOT work because negatives break monotonicity.

Idea. Let $P[i]$ be the prefix sum of the first i elements. A subarray $(j, i]$ sums to k iff $P[i] - P[j] = k$, i.e. $P[j] = P[i] - k$. Scan left to right keeping a hashmap from prefix-sum value to how many indices produced it; at each step add the count of the needed earlier prefix `cur - k`. Seeding the map with $\{0: 1\}$ accounts for subarrays starting at index 0.

```

1 from collections import defaultdict
2
3 def subarray_sum(nums: list[int], k: int) → int:
4     seen = defaultdict(int)
5     seen[0] = 1          # empty prefix: enables subarrays starting at index 0
6     cur = 0
7     count = 0
8     for x in nums:
9         cur += x
10        count += seen[cur - k]  # how many earlier prefixes complete a sum of k
11        seen[cur] += 1
12    return count

```

**** Error-prone spots:****

- Forgetting to initialize `seen[0] = 1`; you'd miss every subarray that starts at index 0.
- Recording `seen[cur] += 1` *before* the lookup, which can falsely count a zero-length subarray when $k = 0$.
- Trying a sliding window — invalid here because negative numbers mean a growing window's sum isn't monotonic.
- Looking up `cur + k` instead of `cur - k`.
- Using a plain dict and indexing a missing key (`KeyError`); use `defaultdict(int)` or `.get(cur - k, 0)`.

Follow-up. Variant LC 974 (subarrays divisible by K) uses the same idea but keys the map on `cur % k` (normalized to be non-negative).

4.5 Valid Sudoku (LC 36)

Problem. Given a partially filled 9x9 board (cells are digits '1'-'9' or '.'), decide whether the current state is valid: no digit repeats within any row, any column, or any of the nine 3x3 boxes. Empty cells impose no constraint; the board need not be solvable.

Optimal — one pass with three sets of seen keys · Time $O(1)$, Space $O(1)$. The board is fixed 9x9, so it's constant work; the point is a single clean pass instead of three separate scans.

Idea. Track seen digits per row, per column, and per box using sets keyed by index. The box index is $(r // 3) * 3 + c // 3$, mapping each cell to one of nine boxes. On encountering a digit, fail immediately if it's already recorded for that row, column, or box.

```

1 def is_valid_sudoku(board: list[list[str]]) → bool:
2     rows = [set() for _ in range(9)]
3     cols = [set() for _ in range(9)]
4     boxes = [set() for _ in range(9)]
5     for r in range(9):
6         for c in range(9):
7             d = board[r][c]
8             if d == '.':
9                 continue
10            b = (r // 3) * 3 + c // 3
11            if d in rows[r] or d in cols[c] or d in boxes[b]:
12                return False
13            rows[r].add(d)

```

```

14         cols[c].add(d)
15         boxes[b].add(d)
16     return True

```

**** Error-prone spots:****

- › Wrong box-index formula — $(r // 3) * 3 + c // 3$ is correct; mixing up which coordinate is divided/multiplied collapses boxes together.
- › Not skipping ' ' and treating empties as a repeated value.
- › Sharing one set across all rows (or all boxes) instead of nine independent sets.
- › Adding to the sets before the membership check, so a digit never appears to conflict with itself — but only fails subtly on the very first duplicate; check first, then add.
- › Assuming the board must be solvable; this problem only validates the current state.

4.6 Find All Anagrams in a String (LC 438)

Problem. Given strings s and p , return the start indices of every substring of s that is an anagram of p (same multiset of characters). Lowercase English letters only. Output indices in any order (ascending is natural).

Optimal — fixed-size sliding window of counts · Time $O(n)$, Space $O(1)$. Beats sorting each window $O(n * m \log m)$; the alphabet is 26 so the count comparison is constant.

Idea. Maintain a count array for p and a count array for the current window of length $\text{len}(p)$ in s . Slide the window one char at a time, incrementing the entering char and decrementing the leaving char; whenever the two count arrays match, the window is an anagram. To avoid an $O(26)$ compare each step you can track a matches counter, but comparing 26-length arrays is already constant.

```

1  from collections import Counter
2
3  def find_anagrams(s: str, p: str) → list[int]:
4      if len(p) > len(s):
5          return []
6      need = Counter(p)
7      window = Counter(s[:len(p)])
8      res = []
9      if window == need:
10         res.append(0)
11     for i in range(len(p), len(s)):
12         window[s[i]] += 1           # char entering on the right
13         left = s[i - len(p)]
14         window[left] -= 1           # char leaving on the left
15         if window[left] == 0:
16             del window[left]       # keep Counter clean so == works
17         if window == need:
18             res.append(i - len(p) + 1)
19     return res

```

**** Error-prone spots:****

- › Leaving zero-count keys in the window Counter; Counter equality treats $\{'a':0\}$ as unequal to a missing key, so you must `del` keys that hit 0.

- › Off-by-one on the appended start index: it is $i - \text{len}(p) + 1$, not i .
- › Forgetting the initial window check before the loop (misses an anagram at index 0).
- › Not guarding $\text{len}(p) > \text{len}(s)$ up front.
- › Rebuilding the window counter from scratch each step (turns $O(n)$ into $O(n*m)$).

Follow-up. LC 567 "Permutation in String" is the boolean version — return True on the first match instead of collecting all starts.

5 Linked Lists

Singly linked lists shine when you manipulate pointers in place: reversal, cycle detection, and merging all run in $O(1)$ extra space. A dummy/sentinel head node and the fast/slow two-pointer technique eliminate most edge-case branching.

```

1 # Shared definition for this section.
2 class ListNode:
3     def __init__(self, val: int = 0, next: "ListNode | None" = None):
4         self.val = val
5         self.next = next

```

5.1 Reverse Linked List (LC 206)

Problem. Given the head of a singly linked list, reverse it and return the new head. The list may be empty.

Optimal — iterative pointer reversal · Time $O(n)$, Space $O(1)$. The recursive version is also $O(n)$ time but uses $O(n)$ stack space.

Idea. Walk the list carrying a prev pointer (initially None). For each node, stash its next, point the node back at prev, then advance prev and cur. When cur falls off the end, prev is the new head. The recursive form reverses the tail first, then flips the link `head.next.next = head`.

```

1 def reverse_list(head: ListNode | None) → ListNode | None:
2     prev = None
3     cur = head
4     while cur:
5         nxt = cur.next    # save before we overwrite
6         cur.next = prev  # reverse the link
7         prev = cur
8         cur = nxt
9     return prev
10
11 def reverse_list_recursive(head: ListNode | None) → ListNode | None:
12     if head is None or head.next is None:
13         return head
14     new_head = reverse_list_recursive(head.next)
15     head.next.next = head  # make the next node point back at us
16     head.next = None      # break the old forward link
17     return new_head

```

**** Error-prone spots:****

- › Overwriting `cur.next` before saving it in `nxt` — you lose the rest of the list.
- › Returning `cur` (which is `None`) instead of `prev` at the end.
- › In the recursive form, forgetting `head.next = None`, which leaves a cycle between the last two nodes.
- › Recursing on the empty list without the `head is None` base case.
- › Returning the original `head` instead of `new_head` from recursion.

5.2 Merge Two Sorted Lists (LC 21)

Problem. Given the heads of two sorted (ascending) linked lists, splice them into one sorted list and return its head. Either or both may be empty.

Optimal — dummy head + tail splice · Time $O(n + m)$, Space $O(1)$. Reuses existing nodes, no allocation beyond one sentinel.

Idea. Use a dummy node to anchor the result so you never special-case the first append. Keep a `tail` pointer; repeatedly attach the smaller of the two current heads and advance that list. When one list is exhausted, attach the entire remaining other list in $O(1)$.



```

1 def merge_two_lists(l1: ListNode | None, l2: ListNode | None) → ListNode | None:
2     dummy = ListNode()
3     tail = dummy
4     while l1 and l2:
5         if l1.val ≤ l2.val:
6             tail.next = l1
7             l1 = l1.next
8         else:
9             tail.next = l2
10            l2 = l2.next
11            tail = tail.next
12    tail.next = l1 if l1 else l2  # append the non-empty remainder
13    return dummy.next

```

**** Error-prone spots:****

- › Forgetting to advance `tail` after splicing, corrupting the chain.
- › Not attaching the leftover list after the loop, dropping the tail.
- › Using `<` instead of `≤` is fine for correctness but breaks stability if that matters.
- › Returning `dummy` instead of `dummy.next`.
- › Allocating new nodes instead of relinking the originals (wastes space).

5.3 Linked List Cycle (LC 141)

Problem. Given the head of a linked list, return `True` if it contains a cycle (some node's next points back to an earlier node), else `False`. Do it with $O(1)$ extra space.

Optimal — Floyd's tortoise and hare · Time $O(n)$, Space $O(1)$. Beats the hash-set-of-visited-nodes approach which uses $O(n)$ space.

Idea. Move a slow pointer one step and a fast pointer two steps per iteration. If there's a cycle, the fast pointer eventually laps the slow one and they meet; if there's no cycle, fast reaches the end (`None`). The relative speed of 1 guarantees they collide inside any loop.

```

1 def has_cycle(head: ListNode | None) → bool:
2     slow = fast = head
3     while fast and fast.next:
4         slow = slow.next
5         fast = fast.next.next
6         if slow is fast:
7             return True
8     return False

```

**** Error-prone spots:****

- › Looping on fast only (not fast and fast.next), causing a NoneType crash when reading fast.next.next.
- › Comparing values (slow.val == fast.val) instead of identity (slow is fast).
- › Initializing slow and fast to different nodes and checking equality before the first move (false positive).
- › Returning True when the loop *ends* (that means no cycle).
- › Forgetting that head may be None.

Follow-up. LC 142 asks for the cycle's entry node: after they meet, reset one pointer to head and advance both one step at a time; they meet at the entry.

5.4 Reorder List (LC 143)

Problem. Given $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$, reorder in place to $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow \dots$. Modify node links only; do not change node values. Return nothing (mutate the list).

Optimal — find middle + reverse second half + merge · Time $O(n)$, Space $O(1)$.

Idea. Three phases. (1) Find the middle with slow/fast pointers. (2) Reverse the second half starting after the middle. (3) Merge the first half and the reversed second half by alternating nodes. Splitting at the right midpoint ensures the two halves differ in length by at most one so the interleave terminates cleanly.

```

1 def reorder_list(head: ListNode | None) → None:
2     if not head or not head.next:
3         return
4     # 1. find middle; for even length, slow ends at the first of the two middles' second
5     slow, fast = head, head
6     while fast.next and fast.next.next:
7         slow = slow.next
8         fast = fast.next.next
9     # 2. reverse the second half
10    second = slow.next
11    slow.next = None          # cut the list into two halves
12    prev = None
13    while second:
14        nxt = second.next
15        second.next = prev
16        prev = second
17        second = nxt
18    # 3. merge two halves, first half is ≥ length of second

```

```

19     first, second = head, prev
20     while second:
21         n1, n2 = first.next, second.next
22         first.next = second
23         second.next = n1
24         first, second = n1, n2

```

**** Error-prone spots:****

- › Not cutting with `slow.next = None`, leaving the first half's tail pointing into the reversed second half (creates a cycle).
- › Using the wrong middle: with `fast.next` and `fast.next.next` the first half is the longer/equal one, which the merge loop relies on.
- › Merge loop driven by `first` instead of `second`; since `second` is the shorter/equal list, looping on it terminates correctly.
- › Forgetting the base case for 0 or 1 nodes.
- › Reordering values instead of relinking nodes (the problem forbids that interpretation when nodes carry more than val).

5.5 Remove Nth Node From End (LC 19)

Problem. Given the head of a list and an integer n ($1 \leq n \leq \text{length}$), remove the n -th node from the end and return the head. Single pass preferred.

Optimal — two-pointer gap + dummy node · Time $O(n)$, Space $O(1)$.

Idea. Put a dummy before head so deleting the actual head is uniform. Advance a `fast` pointer $n + 1$ steps ahead of `slow`; then move both until `fast` is `None`. Now `slow` sits just before the target, so `slow.next = slow.next.next` unlinks it. The $n + 1$ gap (measured from dummy) lands `slow` on the predecessor.

```

1 def remove_nth_from_end(head: ListNode | None, n: int) → ListNode | None:
2     dummy = ListNode(0, head)
3     slow = fast = dummy
4     for _ in range(n + 1): # open a gap of n+1 from the dummy
5         fast = fast.next
6     while fast:
7         slow = slow.next
8         fast = fast.next
9     slow.next = slow.next.next # unlink the n-th-from-end node
10    return dummy.next

```

**** Error-prone spots:****

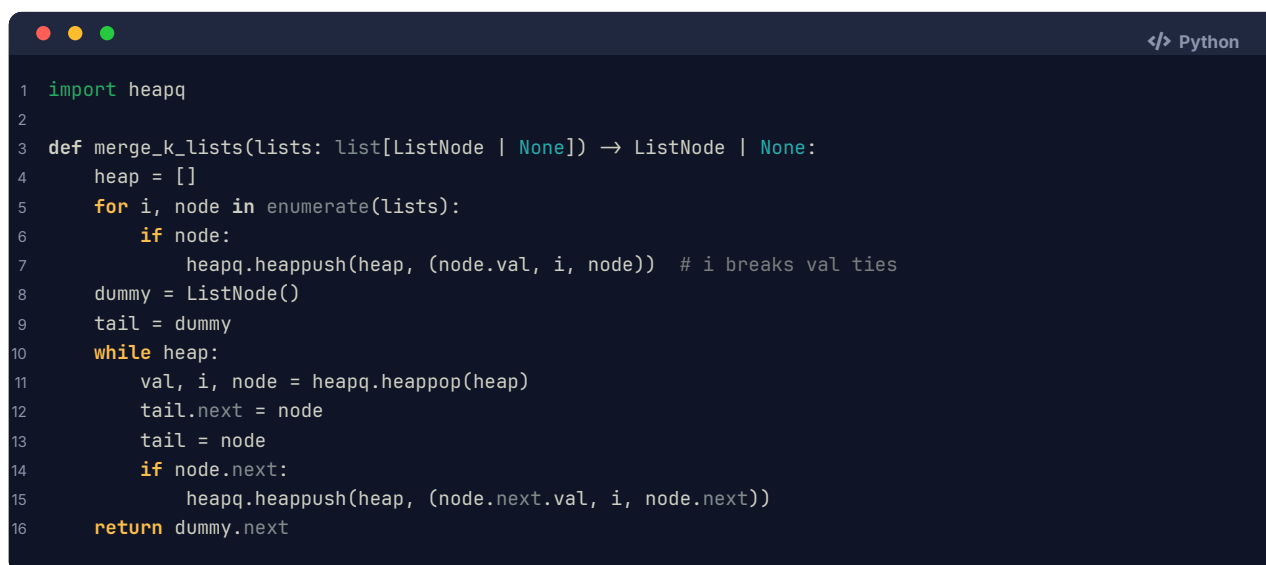
- › Advancing `fast` by n instead of $n + 1$, leaving `slow` on the target rather than its predecessor.
- › Not using a dummy, then crashing or returning the wrong head when removing the first node.
- › Returning `head` instead of `dummy.next` (head may have been the removed node).
- › Reading `slow.next.next` without ensuring the gap math is right (can `NoneType` if off by one).
- › Assuming n could exceed the length; the constraints forbid it, but a defensive check never hurts.

5.6 Merge K Sorted Lists (LC 23)

Problem. Given an array of k sorted linked lists, merge them into one sorted list and return its head. Lists may be empty; k may be 0.

Optimal — min-heap of heads · Time $O(N \log k)$, Space $O(k)$ where N is the total number of nodes. Beats merging sequentially one-by-one which is $O(N^2k)$; divide-and-conquer pairwise merging matches $O(N \log k)$.

Idea. Push the head of each non-empty list into a min-heap keyed by value. Pop the smallest, append it to the result, and push that node's successor. The heap always holds at most k candidates, so each of the N pops/pushes costs $O(\log k)$. Tie-break with the list index so Python never compares `ListNode` objects directly.



```

1 import heapq
2
3 def merge_k_lists(lists: list[ListNode | None]) → ListNode | None:
4     heap = []
5     for i, node in enumerate(lists):
6         if node:
7             heapq.heappush(heap, (node.val, i, node)) # i breaks val ties
8     dummy = ListNode()
9     tail = dummy
10    while heap:
11        val, i, node = heapq.heappop(heap)
12        tail.next = node
13        tail = node
14        if node.next:
15            heapq.heappush(heap, (node.next.val, i, node.next))
16    return dummy.next

```

**** Error-prone spots:****

- Omitting the index tie-breaker: when two nodes share a value, the heap tries to compare `ListNode` objects and raises `TypeError`.
- Pushing `None` heads into the heap (guard with `if node`).
- Forgetting to push the popped node's next, so the rest of that list is dropped.
- Not terminating the merged list — set `tail.next = None` if input nodes might already carry stale next pointers (here they don't, since we overwrite as we go and the final node's next is its own original `None`).
- Returning `dummy` instead of `dummy.next`.

Follow-up. Divide-and-conquer: repeatedly merge list pairs with `merge_two_lists`, halving the count each round — same $O(N \log k)$ with no heap.

6 Stacks & Queues

A stack (LIFO) tracks "the most recent unmatched thing," powering bracket matching and expression evaluation; a monotonic stack/deque maintains a sorted-by-construction window to answer next-greater and range-extreme queries in amortized $O(1)$ per element.

6.1 Valid Parentheses (LC 20)

Problem. Given a string of just `()[]{}` , return `True` if every opening bracket is closed by the matching type in the correct order. Empty string is valid.

Optimal — stack · Time $O(n)$, Space $O(n)$.

Idea. Push every opening bracket. On a closing bracket, the top of the stack must be its matching opener; if the stack is empty or mismatched, fail. A valid string ends with an empty stack. The stack captures the strict nesting order that simple counting cannot.

```
1 def is_valid(s: str) → bool:
2     pairs = {'(': ')', '[': ']', '{': '}'
3     stack = []
4     for ch in s:
5         if ch in pairs:                # closing bracket
6             if not stack or stack.pop() != pairs[ch]:
7                 return False
8         else:                          # opening bracket
9             stack.append(ch)
10    return not stack
```

**** Error-prone spots:****

- › Returning `True` without the final `not stack` check, accepting unclosed openers like `"((("`.
- › Popping an empty stack on a leading closing bracket — guard with `not stack` first.
- › Counting brackets instead of matching types, so `"([)]"` wrongly passes.
- › Mapping openers to closers but then comparing against the wrong direction.
- › Forgetting that the empty string should return `True`.

6.2 Min Stack (LC 155)

Problem. Design a stack supporting `push(x)`, `pop()`, `top()`, and `getMin()` all in $O(1)$. `pop/top/getMin` are only called on a non-empty stack.

Optimal — pair each value with the running min · Time $O(1)$ per op, Space $O(n)$.

Idea. Store on each stack entry both the value and the minimum of the stack up to and including that entry. The new minimum on push is `min(x, current_min)`. Because the min is recorded per level, popping automatically restores the previous minimum with no recomputation.

```
1 class MinStack:
2     def __init__(self):
3         self.stack: list[tuple[int, int]] = [] # (value, min_so_far)
4
5     def push(self, val: int) → None:
6         cur_min = val if not self.stack else min(val, self.stack[-1][1])
7         self.stack.append((val, cur_min))
8
9     def pop(self) → None:
10        self.stack.pop()
11
12    def top(self) → int:
13        return self.stack[-1][0]
```

```

14
15     def getMin(self) → int:
16         return self.stack[-1][1]

```

**** Error-prone spots:****

- › Keeping a single scalar min variable: it can't be restored after popping the current minimum.
- › Computing cur_min from an empty stack — handle the first push specially.
- › Returning the tuple from top() instead of [0], or the value from getMin() instead of [1].
- › A two-stack variant must push onto the min-stack on ties ($\leq$), or popping equal minima corrupts the answer.
- › Forgetting that all four operations must be $O(1)$ — no scanning on getMin.

6.3 Daily Temperatures (LC 739)

Problem. Given temperatures, return an array answer where answer[i] is the number of days until a warmer temperature, or 0 if none exists.

Optimal — monotonic decreasing stack of indices · Time $O(n)$, Space $O(n)$. Beats the $O(n^2)$ "scan forward for each day" approach.

Idea. Keep a stack of indices whose temperatures are still waiting for a warmer day, kept decreasing from bottom to top. For each new day, pop every index colder than today and set its answer to the index gap; then push today. Each index is pushed and popped at most once, giving amortized $O(1)$ per element.

```

1  def daily_temperatures(temperatures: list[int]) → list[int]:
2      answer = [0] * len(temperatures)
3      stack = [] # indices with strictly decreasing temperatures
4      for i, t in enumerate(temperatures):
5          while stack and temperatures[stack[-1]] < t:
6              j = stack.pop()
7              answer[j] = i - j # days waited
8              stack.append(i)
9      return answer

```

**** Error-prone spots:****

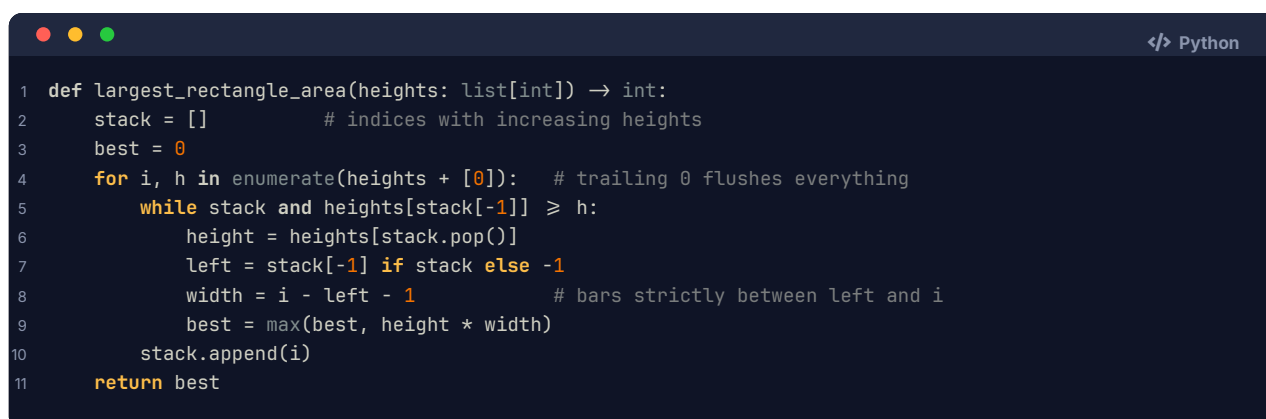
- › Storing temperatures instead of indices, losing the distance needed for the answer.
- › Wrong comparison direction: pop while the stacked temp is strictly less than today's.
- › Using $\leq$ and popping equal temperatures changes "warmer" into "warmer-or-equal" — keep it strict $<$.
- › Setting answer[j] = i instead of the gap i - j.
- › Leaving indices on the stack means their answer stays 0, which is correct — don't post-process them.

6.4 Largest Rectangle in Histogram (LC 84)

Problem. Given heights of bars each of width 1, return the area of the largest axis-aligned rectangle that fits under the histogram outline.

Optimal — monotonic increasing stack · Time $O(n)$, Space $O(n)$. Beats the $O(n^2)$ "expand each bar left/right" approach.

Idea. Maintain a stack of indices with increasing heights. When the current bar is shorter than the stack top, that taller bar can't extend further right, so pop it and compute the rectangle using the popped height; its left boundary is the new stack top (or the start) and its right boundary is the current index. Appending a sentinel height 0 at the end flushes the stack so every bar is resolved.



```

1 def largest_rectangle_area(heights: list[int]) → int:
2     stack = []          # indices with increasing heights
3     best = 0
4     for i, h in enumerate(heights + [0]): # trailing 0 flushes everything
5         while stack and heights[stack[-1]] ≥ h:
6             height = heights[stack.pop()]
7             left = stack[-1] if stack else -1
8             width = i - left - 1          # bars strictly between left and i
9             best = max(best, height * width)
10        stack.append(i)
11    return best

```

**** Error-prone spots:****

- Width formula: it is $i - \text{stack}[-1] - 1$ after popping (left boundary exclusive), or i when the stack is empty — using $i - \text{stack.pop}()$ is wrong.
- Forgetting the trailing sentinel 0, leaving tall bars unresolved on the stack.
- Pushing onto a stack indexed beyond `heights` when the sentinel appends $i = \text{len}(\text{heights})$; that index i is only used as a right boundary, never read from `heights`, so it's safe — but reading `heights[i]` there would crash.
- Using $>$ instead of $\geq$ for the pop test can mishandle equal-height bars (both choices work if widths are computed consistently, but be deliberate).
- Resetting `left` to 0 instead of -1 when the stack empties, under-counting the leftmost width by one.

Follow-up. LC 85 "Maximal Rectangle" reduces row-by-row to this histogram problem by treating accumulated column heights as a histogram per row.

6.5 Sliding Window Maximum (LC 239)

Problem. Given `nums` and window size k , return the maximum of each contiguous window of length k as the window slides left to right. $1 \leq k \leq \text{len}(\text{nums})$.

Optimal — monotonic decreasing deque of indices · Time $O(n)$, Space $O(k)$. Beats the heap solution's $O(n \log n)$ and the brute force $O(n \cdot k)$.

Idea. Keep a deque of indices whose values are decreasing front-to-back; the front always holds the current window's maximum. Before adding a new index, pop from the back every index with a value $\leq$ the incoming one (they can never be the max again), and pop from the front any index that has slid out of the window. Each index enters and leaves the deque once $\rightarrow$ amortized $O(1)$.

```

1 from collections import deque
2
3 def max_sliding_window(nums: list[int], k: int) → list[int]:
4     dq = deque()    # indices, values decreasing front → back
5     res = []
6     for i, x in enumerate(nums):
7         while dq and nums[dq[-1]] ≤ x:    # drop smaller tail values
8             dq.pop()
9         dq.append(i)
10        if dq[0] == i - k:                # front index left the window
11            dq.popleft()
12        if i ≥ k - 1:                    # first full window reached
13            res.append(nums[dq[0]])
14    return res

```

**** Error-prone spots:****

- › Evicting the out-of-window front by value comparison instead of index (`dq[0] == i - k`).
- › Storing values instead of indices, making window-expiry checks impossible.
- › Recording results before the first full window forms (`i ≥ k - 1`).
- › Using `<` instead of `≤` when popping the tail keeps stale equal values around — both are correct for the max but `≤` keeps the deque smaller.
- › Reading `dq[0]` when the deque could be empty — here it never is right after an append, but reorder carelessly and it breaks.

6.6 Evaluate Reverse Polish Notation (LC 150)

Problem. Evaluate an arithmetic expression in Reverse Polish (postfix) notation given as a token list. Operators are `+` `-` `*` `/`; operands are integers. Division truncates toward zero. The input is always a valid RPN expression.

Optimal — stack · Time $O(n)$, Space $O(n)$.

Idea. Scan left to right pushing operands. On an operator, pop the top two operands (the second-popped is the left operand), apply the operator, and push the result. A valid RPN expression leaves exactly one value on the stack. The only subtlety is integer division truncating toward zero, which Python's `//` does not do for negatives.

```

1 def eval_rpn(tokens: list[str]) → int:
2     ops = {'+', '-', '*', '/'}
3     stack = []
4     for tok in tokens:
5         if tok in ops:
6             b = stack.pop()
7             a = stack.pop()    # a is the left operand
8             if tok == '+':
9                 stack.append(a + b)
10            elif tok == '-':
11                stack.append(a - b)
12            elif tok == '*':
13                stack.append(a * b)
14            else:
15                stack.append(int(a / b))    # truncate toward zero, not floor
16    else:

```

```

17         stack.append(int(tok))
18     return stack[0]

```

**** Error-prone spots:****

- › Operand order: the first pop is the right operand b, the second is the left a; reversing them breaks - and /.
- › Using `a // b` for division — Python floors toward negative infinity, e.g. `-7 // 2 = -4`, but the problem wants `int(-7 / 2) = -3`.
- › Detecting numbers via `tok.isdigit()`, which fails on negative literals like `"-3"`; test membership in the operator set instead.
- › Forgetting to `int()`-convert string operands before pushing.
- › Returning the whole stack instead of its single remaining element.

7 Trees

Recursive structures where most problems reduce to a DFS (pre/in/post-order) or a level-by-level BFS; the trick is choosing the traversal whose visit order matches the invariant you need.

A single node definition is reused throughout this section:

```

1 class TreeNode:
2     def __init__(self, val=0, left=None, right=None):
3         self.val = val
4         self.left = left
5         self.right = right

```

7.1 Maximum Depth of Binary Tree (LC 104)

Problem. Given the root of a binary tree, return its maximum depth — the number of nodes along the longest path from the root down to the farthest leaf. An empty tree has depth 0; a single node has depth 1.

Optimal — post-order DFS · Time $O(n)$, Space $O(h)$. $O(n)$ is unavoidable since every node must be inspected; space is the recursion stack of height h ($O(n)$ worst case for a skewed tree, $O(\log n)$ balanced). Beats nothing asymptotically but is the cleanest formulation.

Idea. The depth of a tree is 1 plus the maximum of the depths of its two subtrees. The recursion bottoms out at the null child returning 0, so a leaf returns `1 + max(0, 0) = 1`.

```

1 def maxDepth(root: TreeNode) → int:
2     if not root:
3         return 0
4     return 1 + max(maxDepth(root.left), maxDepth(root.right))

```

**** Error-prone spots:****

- › Returning 0 vs 1 at the base case: the null node returns 0, not the leaf. Returning at the leaf

instead forces awkward null-child checks.

- › Off-by-one: forgetting the 1 + and summing child depths, which counts edges or worse.
- › Confusing depth (node count) with the number of edges (which is depth - 1); read the problem's definition.
- › For very deep skewed trees ($n \sim 10^4+$), Python's default recursion limit (1000) can overflow — an explicit stack/BFS is safer in that regime.

Follow-up. Minimum depth (LC 111) is subtly different: a node with only one child must NOT count the missing side as depth 0, or you'll report a too-shallow value; recurse only into existing children.

7.2 Invert Binary Tree (LC 226)

Problem. Given the root of a binary tree, swap every node's left and right child (mirror the tree) and return the root. Handle the empty tree.

Optimal — DFS swap · Time $O(n)$, Space $O(h)$. Every node is visited once; recursion stack is $O(h)$. No faster approach exists since the whole tree must be touched.

Idea. Inverting a tree means inverting both subtrees and then swapping the two child pointers. The order (swap before or after recursing) does not matter as long as you capture both children before overwriting.

```

1 def invertTree(root: TreeNode) -> TreeNode:
2     if not root:
3         return None
4     root.left, root.right = invertTree(root.right), invertTree(root.left)
5     return root

```

**** Error-prone spots:****

- › Overwriting `root.left` before reading it, then recursing into the now-clobbered pointer — the tuple assignment above avoids this by evaluating the RHS fully first.
- › Forgetting to return `root` (the function must hand back the same root reference).
- › Doing `invertTree(root.left)` and `invertTree(root.right)` but never swapping the pointers — that recurses without mirroring.
- › An iterative BFS/DFS variant must enqueue children and swap; forgetting to enqueue the (now swapped) children skips subtrees.

Follow-up. Symmetric Tree (LC 101) is the verification analogue: check that the left subtree is a mirror of the right without modifying anything, comparing `left.left` to `right.right` and `left.right` to `right.left`.

7.3 Same Tree (LC 100)

Problem. Given roots `p` and `q` of two binary trees, return `True` iff they are structurally identical and every corresponding node has the same value. Two empty trees are equal.

Optimal — parallel DFS · Time $O(\min(n, m))$, Space $O(\min(h))$. Traversal stops at the first mismatch, so cost is bounded by the smaller tree. Space is the recursion depth.

Idea. Two trees are the same iff their roots match in existence and value and their respective left and right subtrees are recursively the same. Short-circuit evaluation lets a structural

mismatch (one null, one not) be caught before dereferencing.

```

1 def isSameTree(p: TreeNode, q: TreeNode) → bool:
2     if not p and not q:
3         return True
4     if not p or not q or p.val ≠ q.val:
5         return False
6     return isSameTree(p.left, q.left) and isSameTree(p.right, q.right)

```

**** Error-prone spots:****

- › Checking `p.val ≠ q.val` before confirming both are non-null raises `AttributeError` on a `None`; the ordered guards above prevent this.
- › Treating two empty trees as unequal – they are equal.
- › Comparing only values and not structure: a node present in one tree and absent in the other must fail.
- › Using `=` on the nodes themselves (identity) instead of recursing on values and shape.

Follow-up. Subtree of Another Tree (LC 572): for each node of the main tree, run `isSameTree` against the candidate subtree; $O(n*m)$ naive, or $O(n+m)$ via tree serialization + KMP/string search.

7.4 Binary Tree Level Order Traversal (LC 102)

Problem. Return the node values level by level, top to bottom, left to right, as a list of lists (one inner list per level). Empty tree returns `[]`.

Optimal – BFS with a queue · Time $O(n)$, Space $O(n)$. Each node is enqueued/dequeued once; the queue holds up to one full level, $O(n)$ worst case (the widest level).

Idea. Process the tree breadth-first, but snapshot the queue's size at the start of each level so you dequeue exactly the nodes of the current level before their children pollute the queue. That fixed count cleanly separates levels.

```

1 from collections import deque
2
3 def levelOrder(root: TreeNode) → list[list[int]]:
4     if not root:
5         return []
6     result, q = [], deque([root])
7     while q:
8         level = []
9         for _ in range(len(q)):           # freeze count before enqueueing children
10            node = q.popleft()
11            level.append(node.val)
12            if node.left:
13                q.append(node.left)
14            if node.right:
15                q.append(node.right)
16        result.append(level)
17    return result

```

**** Error-prone spots:****

- › Iterating for `_ in range(len(q))` but calling `len(q)` inside the loop or after appending children — capture the size once per level.
- › Using a list with `pop(0)` instead of `deque.popleft()` — `pop(0)` is $O(n)$, degrading the whole thing to $O(n^2)$.
- › Forgetting the empty-tree guard, which produces `[[] ]` or an error instead of `[]`.
- › Appending children before finishing the current level's reads (only a bug if you mismanage the size snapshot).

Follow-up. Zigzag level order (LC 103): same BFS, but reverse every other level's list (or append-left into a deque) for the alternating direction.

7.5 Validate Binary Search Tree (LC 98)

Problem. Determine whether a binary tree is a valid BST: for every node, all values in its left subtree are strictly less, and all in its right subtree are strictly greater. Duplicates are not allowed under the standard definition.

Optimal — bounded DFS · Time $O(n)$, Space $O(h)$. Every node checked once against an inherited (low, high) open interval. The naive bug-prone version only compares a node to its immediate children, which is wrong.

Idea. A node is valid only if its value lies strictly inside the open interval (low, high) it inherits from ancestors. Recursing left tightens the upper bound to the node's value; recursing right tightens the lower bound. This propagates ancestor constraints that a parent-only check misses.

```

1 def isValidBST(root: TreeNode) → bool:
2     def valid(node, low, high):
3         if not node:
4             return True
5         if not (low < node.val < high):
6             return False
7         return valid(node.left, low, node.val) and valid(node.right, node.val, high)
8     return valid(root, float('-inf'), float('inf'))

```

**** Error-prone spots:****

- › The classic bug: only checking `node.left.val < node.val < node.right.val`. A deep descendant can violate an ancestor's bound while satisfying its parent — e.g. root 5, right child 7, with 7's left child = 3 must fail.
- › Using $\leq$ instead of $<$ and wrongly admitting duplicates (standard LC definition forbids equal values).
- › Initializing bounds with a fixed int like $\pm 2^{31}$ instead of $\pm \text{inf}$ — node values can hit those exact extremes (INT_MIN/INT_MAX), causing false negatives.
- › The in-order alternative must track the previous value and fail on a non-strict increase; forgetting to update `prev` or mishandling the first node breaks it.

Follow-up. The in-order traversal of a valid BST yields a strictly increasing sequence; an equivalent $O(n)/O(h)$ solution walks in-order and verifies each value exceeds the previous.

7.6 Lowest Common Ancestor of a BST (LC 235)

Problem. Given a BST and two nodes p and q known to exist in it, return their lowest common ancestor — the deepest node that has both as descendants (a node is a descendant of itself).

Optimal — BST walk · Time $O(h)$, Space $O(1)$ iterative. Exploiting ordering gives $O(h)$ ($O(\log n)$ balanced) versus the $O(n)$ general-tree LCA. Iterative uses $O(1)$ extra space.

Idea. The LCA is the first (highest) node where p and q fall on different sides — or where one of them equals the current node. If both values are less than the current node, the answer lies left; if both greater, it lies right; otherwise the current node is the split point and thus the LCA.

```
1 def lowestCommonAncestor(root: TreeNode, p: TreeNode, q: TreeNode) → TreeNode:
2     node = root
3     while node:
4         if p.val < node.val and q.val < node.val:
5             node = node.left
6         elif p.val > node.val and q.val > node.val:
7             node = node.right
8         else:
9             return node          # split point, or one of them is node itself
```

**** Error-prone spots:****

- › Forgetting the equality case: when node.val equals $p.val$ or $q.val$, that node IS the LCA (one is an ancestor of the other) — the else branch handles it, but an explicit `</>`-only set of conditions can skip it.
- › Using $\leq / \geq$ and walking past the correct split node.
- › Comparing node references where you should compare `.val` (works only if p, q are the actual tree nodes; comparing vals is safest).
- › Assuming the general-tree LCA algorithm here — it works but wastes the BST property and is $O(n)$.

Follow-up. LCA of a general binary tree (LC 236): no ordering, so DFS returns a node if it (or one of its subtrees) contains p or q ; the node where both sides return non-null is the LCA.

7.7 Kth Smallest Element in a BST (LC 230)

Problem. Given a BST and integer k (1-indexed), return the k -th smallest value. $1 \leq k \leq$ number of nodes.

Optimal — in-order traversal with early stop · Time $O(h + k)$, Space $O(h)$. An in-order walk yields values in sorted order; stopping at the k -th visited node costs $O(h + k)$ rather than $O(n)$ for a full traversal or $O(n \log n)$ for sorting.

Idea. In-order traversal of a BST produces ascending values. Maintain a counter; the k -th node popped in in-order order is the answer. An iterative stack lets you bail out the instant the counter hits k .

```
1 def kthSmallest(root: TreeNode, k: int) → int:
2     stack, node = [], root
3     while stack or node:
4         while node:          # descend to the leftmost unvisited node
```

```

5         stack.append(node)
6         node = node.left
7     node = stack.pop()
8     k -= 1
9     if k == 0:
10        return node.val
11    node = node.right        # then explore the right subtree

```

**** Error-prone spots:****

- Off-by-one with 1-indexing: decrement k after popping and return when it reaches 0, not 1.
- Forgetting to go right after popping — you'd re-descend the same left spine forever or miss right subtrees.
- The inner while node push-left loop must run before each pop; merging the loops wrongly skips nodes.
- A full recursive in-order that collects all n values wastes time/space; the early stop is the point of the optimization.

Follow-up. If the tree is modified often and many such queries are made, augment each node with its subtree size; then each query is $O(h)$ by descending using subtree counts.

7.8 Serialize and Deserialize Binary Tree (LC 297)

Problem. Design `serialize(root) → str` and `deserialize(str) → root` so that `deserialize(serialize(t))` reconstructs the original tree. Values may be arbitrary ints; the tree may be empty; structure (including null children) must be preserved.

Optimal — preorder with null markers · Time $O(n)$, Space $O(n)$. Each node and each null slot is emitted/consumed once. Preorder plus explicit nulls uniquely determines the tree without a second traversal.

Idea. Preorder (root, left, right) with a sentinel for every missing child fully encodes the shape: when rebuilding, the very next token is always the root of the subtree we're about to construct, then we recursively build left then right, consuming nulls as we hit them. An iterator over the token stream keeps the consumption position implicit.

```

1 class Codec:
2     def serialize(self, root: TreeNode) → str:
3         out = []
4         def dfs(node):
5             if not node:
6                 out.append('#')        # null marker
7                 return
8             out.append(str(node.val))
9             dfs(node.left)
10            dfs(node.right)
11        dfs(root)
12        return ','.join(out)
13
14    def deserialize(self, data: str) → TreeNode:
15        vals = iter(data.split(','))
16        def build():
17            v = next(vals)

```

```

18         if v == '#':
19             return None
20         node = TreeNode(int(v))
21         node.left = build()      # preorder: left before right
22         node.right = build()
23         return node
24     return build()

```

**** Error-prone spots:****

- › Omitting null markers: preorder alone (without nulls) is NOT enough to reconstruct an arbitrary binary tree; you must record the missing children.
- › Building right before left in deserialize, which mirrors the tree — order must match serialization.
- › Using a shared index that you forget to advance, or passing it by value instead of reference; the iter/next pattern sidesteps manual index bookkeeping.
- › Choosing a delimiter or null marker that can collide with real values (e.g. negative numbers vs a '-' split). Comma + '#' is safe for integer payloads.
- › Mishandling the empty tree: serialize(None) must round-trip back to None (here it yields '#').

Follow-up. A BST can be serialized as preorder WITHOUT null markers (LC 449), since the BST ordering lets you infer structure from value bounds, yielding a more compact encoding.

7.9 Diameter of Binary Tree (LC 543)

Problem. Return the length of the longest path between any two nodes, measured in edges. The path need not pass through the root. Empty tree has diameter 0.

Optimal — post-order height with global max · Time $O(n)$, Space $O(h)$. A single post-order pass computes heights while updating the best diameter, beating the $O(n^2)$ approach of computing height independently at every node.

Idea. The longest path through a given node equals leftHeight + rightHeight (in edges). Compute each node's height bottom-up, and at each node update a global maximum with the sum of its two child heights. The function returns height to the parent while the side effect records the diameter.

```

1 def diameterOfBinaryTree(root: TreeNode) → int:
2     best = 0
3     def height(node):
4         nonlocal best
5         if not node:
6             return 0          # height in edges of an empty subtree
7         lh = height(node.left)
8         rh = height(node.right)
9         best = max(best, lh + rh) # path through this node, in edges
10        return 1 + max(lh, rh)
11    height(root)
12    return best

```

**** Error-prone spots:****

- › Returning the diameter from the recursion instead of the height — the function must return height; the diameter is the accumulated best.
- › Mixing edge-counting and node-counting: diameter here is edges, so the through-node path is $lh + rh$ (not $+ 1$), while the returned height is $1 + \max(lh, rh)$.
- › Forgetting `nonlocal best` (or using a one-element list), so updates are lost.
- › Computing `height(node)` separately at every node — correct but $O(n^2)$; the single-pass side effect is the optimization.

Follow-up. Binary Tree Maximum Path Sum (LC 124) generalizes this: track a global max path sum, but clamp negative subtree contributions to 0 since you may choose to exclude a subtree.

8 Heaps / Priority Queues

A heap maintains the min (or max) element at the root in $O(\log n)$ per push/pop, making it the tool of choice for "top k", streaming order statistics, and repeatedly extracting the current extreme. Python's `heapq` is a binary min-heap over a list.

8.1 Kth Largest Element in an Array (LC 215)

Problem. Given an unsorted array `nums` and integer `k`, return the `k`-th largest element in sorted order (the `k`-th largest VALUE, allowing duplicates — not the `k`-th distinct value). $1 \leq k \leq \text{len}(\text{nums})$.

Optimal — QuickSelect · Time $O(n)$ average / $O(n^2)$ worst, Space $O(1)$. QuickSelect beats the $O(n \log k)$ min-heap and $O(n \log n)$ full sort on average. The min-heap-of-size-`k` approach is preferred when `n` is huge or streaming and you want guaranteed $O(n \log k)$.

Idea (heap). Keep a min-heap of the `k` largest elements seen. Push each value; once the heap exceeds size `k`, pop the smallest. The root is then the `k`-th largest. **Idea (QuickSelect).** Partition around a pivot; the pivot lands at its final sorted index — recurse only into the side containing the target index (`k`-th largest = index `n-k` in ascending order).

```

1 import heapq, random
2
3 def findKthLargest_heap(nums: list[int], k: int) → int:
4     heap = []
5     for x in nums:
6         heapq.heappush(heap, x)
7         if len(heap) > k:
8             heapq.heappop(heap)      # drop the smallest, keep k largest
9     return heap[0]                  # min of the k largest = k-th largest
10
11 def findKthLargest(nums: list[int], k: int) → int:
12     target = len(nums) - k          # k-th largest is this index when ascending-sorted
13     def quickselect(lo, hi):
14         pivot = nums[random.randint(lo, hi)] # random pivot avoids  $O(n^2)$  on sorted input
15         left, mid, right = [], [], []
16         for x in nums[lo:hi + 1]:
17             (left if x < pivot else right if x > pivot else mid).append(x)
18         if target < lo + len(left):
19             return quickselect(lo, lo + len(left) - 1)
20         if target ≥ lo + len(left) + len(mid):
21             return quickselect(lo + len(left) + len(mid), hi)

```

```

22     return pivot                # target falls within the pivot's equal block
23     return quickselect(0, len(nums) - 1)

```

**** Error-prone spots:****

- › Index confusion: k -th LARGEST corresponds to ascending index $n-k$, not k or $k-1$.
- › Heap version: comparing $>$ vs $\geq$ when trimming — push first, then pop only when size strictly exceeds k .
- › QuickSelect with a fixed (non-random) pivot degrades to $O(n^2)$ on already-sorted/adversarial input; randomize or use median-of-medians for worst-case $O(n)$.
- › Mishandling duplicates: a clean three-way partition (less / equal / greater) avoids infinite recursion when many values equal the pivot.
- › The above QuickSelect rewrites partitions into new lists for clarity; an in-place Lomuto/Hoare partition is more memory-efficient but trickier to get right.

Follow-up. For a continuous stream where k is fixed, "Kth Largest in a Stream" (LC 703) just maintains the size- k min-heap permanently and returns its root after each add.

8.2 K Closest Points to Origin (LC 973)

Problem. Given a list of points $[x, y]$ and integer k , return the k points closest to the origin by Euclidean distance, in any order. Ties may be broken arbitrarily.

Optimal — max-heap of size k · Time $O(n \log k)$, Space $O(k)$. Beats sorting all points ($O(n \log n)$) when $k \ll n$. QuickSelect on squared distance gives $O(n)$ average if only the set (not order) is required.

Idea. Compare by squared distance $xx + yy$ (the monotonic square root is unnecessary and avoids floats). Maintain a max-heap of size k by pushing the negated distance; when it exceeds k , pop the farthest. What remains are the k nearest.

```

1 import heapq
2
3 def kClosest(points: list[list[int]], k: int) → list[list[int]]:
4     heap = []                # max-heap via negated distances
5     for x, y in points:
6         d = x * x + y * y    # squared distance: no sqrt, stays integer
7         heapq.heappush(heap, (-d, x, y))
8         if len(heap) > k:
9             heapq.heappop(heap)  # evict the current farthest
10    return [[x, y] for _, x, y in heap]

```

**** Error-prone spots:****

- › Taking an actual sqrt — pointless work and introduces floating-point error; compare squared distances.
- › Sign errors when simulating a max-heap with Python's min-heap: negate the distance on push and ignore it on output.
- › Trimming with $>$ vs $\geq$: keep exactly k , so pop only when size exceeds k .
- › Putting an unorderable object as the second tuple element when distances tie — including coordinates (as above) makes tuples comparable and avoids "TypeError: < not supported".

between dicts/lists”.

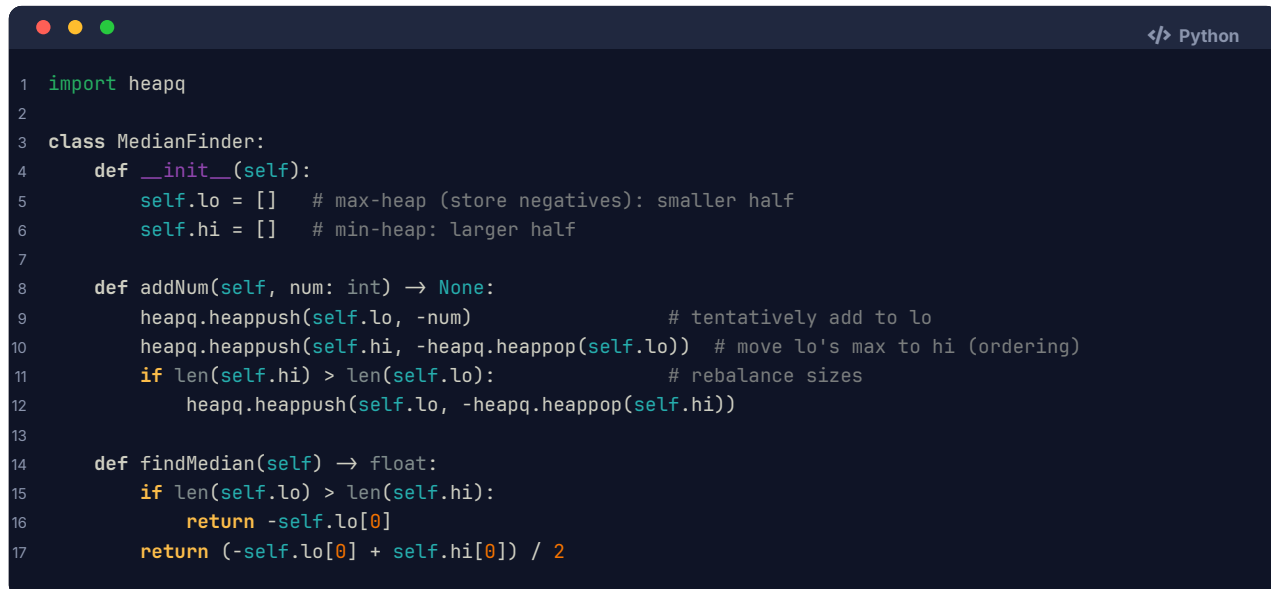
Follow-up. If only the set of k points is needed (not sorted by distance), QuickSelect partitions points by squared distance in $O(n)$ average and returns the first k .

8.3 Find Median from Data Stream (LC 295)

Problem. Design a structure supporting `addNum(int)` to ingest a stream and `findMedian()` $\rightarrow$ `float` returning the median of all values so far. With an even count the median is the average of the two middle values.

Optimal – two balanced heaps · Time $O(\log n)$ add / $O(1)$ median, Space $O(n)$. Beats re-sorting ($O(n \log n)$ per query) and insertion into a sorted list ($O(n)$ per add).

Idea. Keep a max-heap `lo` of the smaller half and a min-heap `hi` of the larger half. Maintain the invariants `len(lo) == len(hi)` or `len(lo) == len(hi)+1`, and every element of `lo` $\leq$ every element of `hi`. The median is then `lo`'s top (odd total) or the average of the two tops (even total). Each add pushes-then-rebalances to preserve both ordering and size.



```

1 import heapq
2
3 class MedianFinder:
4     def __init__(self):
5         self.lo = [] # max-heap (store negatives): smaller half
6         self.hi = [] # min-heap: larger half
7
8     def addNum(self, num: int) → None:
9         heapq.heappush(self.lo, -num) # tentatively add to lo
10        heapq.heappush(self.hi, -heapq.heappop(self.lo)) # move lo's max to hi (ordering)
11        if len(self.hi) > len(self.lo): # rebalance sizes
12            heapq.heappush(self.lo, -heapq.heappop(self.hi))
13
14    def findMedian(self) → float:
15        if len(self.lo) > len(self.hi):
16            return -self.lo[0]
17        return (-self.lo[0] + self.hi[0]) / 2

```

**** Error-prone spots:****

- › Pushing directly into the “correct” half by comparing to the current median — this can violate the cross-heap ordering; the push-to-lo, pop-to-hi, rebalance dance guarantees both invariants.
- › Sign mistakes on the max-heap (`lo` stores negated values): negate on push and on read.
- › Allowing the size difference to exceed 1, which makes `findMedian` read the wrong element.
- › Integer division for the even case in Python 2 style — use `/ 2` for a float, and remember the result can be a non-integer like 2.5.

Follow-up. If all values lie in a small fixed range, a bucket/count array gives $O(1)$ add and $O(\text{range})$ median; if values arrive sorted, a single pointer suffices.

8.4 Task Scheduler (LC 621)

Problem. Given a list of CPU tasks (chars) and a cooldown n , where the same task must be separated by at least n intervals, return the minimum number of intervals (including idle slots) to finish all tasks. Tasks of different types can run back-to-back.

Optimal — greedy frequency formula · Time $O(t)$ (t = number of tasks), Space $O(1)$. Counting frequencies is $O(t)$; the closed-form bypasses any simulation. A max-heap + cooldown queue simulation also works in $O(t \log 26)$ but the formula is $O(1)$ after counting.

Idea. The most frequent task dictates the skeleton: place its `maxFreq` copies with n -length gaps, forming $(\text{maxFreq} - 1)$ full frames of size $(n + 1)$ plus a final partial frame holding all tasks that share the max frequency. The answer is $\max(t, (\text{maxFreq} - 1) * (n + 1) + \text{countOfMaxFreq})$, where t covers the case of so many distinct tasks that no idling is ever needed.

```

1 from collections import Counter
2
3 def leastInterval(tasks: list[str], n: int) → int:
4     counts = Counter(tasks)
5     max_freq = max(counts.values())
6     num_max = sum(1 for c in counts.values() if c == max_freq)
7     # skeleton of (max_freq-1) frames of size (n+1), plus the trailing max-freq tasks
8     frame = (max_freq - 1) * (n + 1) + num_max
9     return max(len(tasks), frame)

```

**** Error-prone spots:****

- Forgetting the `max(len(tasks), ...)`: when there are many distinct task types, the frames fill completely with no idle slots and the formula can underestimate; `len(tasks)` corrects it.
- Counting only one max-frequency task when several tie — `num_max` must count all tasks at the maximum frequency (they each occupy a slot in the final frame).
- Off-by-one in the frame: it's $(\text{max_freq} - 1)$ full gaps of size $(n + 1)$, not `max_freq` frames.
- Misreading n : it's the cooldown count between identical tasks, so the frame width is $n + 1$ (the task itself plus n following slots).

Follow-up. If you must output an actual valid schedule (not just the count), use the max-heap + cooldown-queue simulation: pop the highest-remaining-count task each tick, push it onto a waiting queue with its ready-time, and release tasks back when their cooldown elapses.

8.5 Merge K Sorted Lists (LC 23)

See Linked Lists — uses a min-heap over list heads.

9 Tries (Prefix Trees)

A trie stores strings along a tree of characters so that shared prefixes share nodes, giving $O(L)$ insert/lookup independent of the number of stored words and making prefix queries natural. Each node typically holds a children map and an end-of-word flag.

9.1 Implement Trie (LC 208)

Problem. Implement a Trie with `insert(word)`, `search(word) → bool` (exact word present), and `startsWith(prefix) → bool` (any stored word has this prefix). Words consist of lowercase letters.

Optimal — character tree · Time $O(L)$ per op, Space $O(\text{total chars})$. L is the word/prefix length; each operation walks one node per character. Beats storing words in a set, which gives $O(L)$ exact lookup but cannot answer prefix queries efficiently.

Idea. Each node maps a character to a child node and carries a boolean marking whether a word ends there. `insert` creates missing children along the path; `search` walks the path and checks the end flag; `startsWith` walks the path and only checks the path exists. The end flag is what distinguishes a stored word from a mere prefix.

```

1 class TrieNode:
2     def __init__(self):
3         self.children = {}
4         self.is_end = False
5
6 class Trie:
7     def __init__(self):
8         self.root = TrieNode()
9
10    def insert(self, word: str) → None:
11        node = self.root
12        for ch in word:
13            if ch not in node.children:
14                node.children[ch] = TrieNode()
15            node = node.children[ch]
16        node.is_end = True
17
18    def search(self, word: str) → bool:
19        node = self._walk(word)
20        return node is not None and node.is_end
21
22    def startsWith(self, prefix: str) → bool:
23        return self._walk(prefix) is not None
24
25    def _walk(self, s: str):
26        node = self.root
27        for ch in s:
28            if ch not in node.children:
29                return None
30            node = node.children[ch]
31        return node

```

**** Error-prone spots:****

- Conflating `search` and `startsWith`: `search` MUST also verify `is_end`; without it, searching a prefix that isn't a full word wrongly returns `True`.
- Forgetting to set `is_end = True` at the end of `insert`, so no word is ever "found".
- Sharing a single mutable default like `children={}` across nodes (a class-level mutable default) — always create a fresh dict per node in `__init__`.
- Returning the root node for the empty string and reporting `startsWith("")` inconsistently; decide the convention (empty prefix matches anything).

Follow-up. Add Word with wildcard '.' (LC 211, "Add and Search Word"): search must branch with DFS over all children when it hits a '.', exploring every possible match.

9.2 Word Search II (LC 212)

Problem. Given an $m \times n$ board of letters and a list of words, return all words from the list that can be formed by sequentially adjacent (up/down/left/right) cells, where each cell is used at most once per word. Each word may appear at most once in the output.

Optimal — trie + DFS backtracking · Time $\sim O(mn4 \cdot 3^{(L-1)})$, Space $O(\text{total chars})$. Building a trie of all words lets one grid DFS prune across all words simultaneously, vastly beating running Word Search I once per word (which re-walks the board for every word).

Idea. Insert every word into a trie. DFS from each cell, descending the trie in lockstep with the path on the board; if the current letter isn't a trie child, prune immediately. When a node's word marker is set, collect that word. Mark visited cells (e.g. swap to '#') and restore on backtrack; deleting found leaves from the trie keeps the search shrinking.

```

1 def findWords(board: list[list[str]], words: list[str]) → list[str]:
2     root = {}
3     for w in words:
4         node = root
5         for ch in w:
6             node = node.setdefault(ch, {})
7             node['$'] = w
8
9     rows, cols = len(board), len(board[0])
10    found = []
11
12    def dfs(r, c, node):
13        ch = board[r][c]
14        nxt = node.get(ch)
15        if nxt is None:
16            return
17        word = nxt.pop('$', None)
18        if word is not None:
19            found.append(word)
20            board[r][c] = '#'
21            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
22                nr, nc = r + dr, c + dc
23                if 0 ≤ nr < rows and 0 ≤ nc < cols and board[nr][nc] ≠ '#':
24                    dfs(nr, nc, nxt)
25            board[r][c] = ch
26            if not nxt:
27                node.pop(ch, None)
28
29    for r in range(rows):
30        for c in range(cols):
31            dfs(r, c, root)
32    return found

```

**** Error-prone spots:****

- Not de-duplicating results: pop the word marker ('\$') when found, so the same word isn't added twice from different paths.
- Forgetting to restore board[r][c] after recursing — corrupts the board for subsequent starts.
- Checking the visited mark before recursing vs after reading the letter: read the cell's letter

first to advance the trie, then guard neighbors against '#'.

- ▶ Running plain Word Search per word ($O(W * m * n * 4^L)$) — the trie is the whole point; it shares prefix work and prunes globally.
- ▶ Leaf pruning (`node.pop(ch)` when a subtree is exhausted) is an important optimization for large word lists; skipping it still gives correct but slower results.

Follow-up. Word Search I (LC 79) is the single-word case: skip the trie and DFS the board directly matching the one target word.

9.3 Replace Words (LC 648)

Problem. Given a dictionary of roots and a sentence, replace every word that has a dictionary root as a prefix with the shortest such root. Words not derivable from any root are left unchanged. Return the resulting sentence.

Optimal — trie of roots · Time $O(D + S)$, Space $O(D)$. D is total root characters, S total sentence characters. Building a trie and matching each word against it beats checking every prefix of every word against a set ($O(S * \text{maxLen})$ with repeated substring hashing) and naturally finds the SHORTEST root by stopping at the first end-of-root node.

Idea. Insert all roots into a trie. For each word, walk down the trie character by character; the moment you reach a node marking the end of a root, that prefix is the shortest matching root — replace and stop. If you fall off the trie before hitting any root end, keep the word as is.

```

1 class TrieNode:
2     def __init__(self):
3         self.children = {}
4         self.is_end = False
5
6 def replaceWords(dictionary: list[str], sentence: str) → str:
7     root = TrieNode()
8     for w in dictionary:
9         node = root
10        for ch in w:
11            node = node.children.setdefault(ch, TrieNode())
12        node.is_end = True
13
14    def shortest_root(word: str) → str:
15        node = root
16        prefix = []
17        for ch in word:
18            if ch not in node.children:
19                return word          # no root is a prefix
20            node = node.children[ch]
21            prefix.append(ch)
22            if node.is_end:
23                return ''.join(prefix) # first (shortest) root end wins
24        return word                  # word itself shorter than any root, or equals one
25
26    return ' '.join(shortest_root(w) for w in sentence.split())

```

**** Error-prone spots:****

- ▶ Returning the longest match: you must stop at the FIRST `is_end` to get the shortest root.
- ▶ Forgetting the no-match case — words with no root prefix stay unchanged.
- ▶ Rebuilding the prefix string per character (fine here) vs slicing the word; either works, but

mixing up the length is a common slip.

- › Splitting/joining on the wrong whitespace — use `split()/' '.join()` to normalize single spaces as the problem assumes.

Follow-up. If roots can be inserted/queried dynamically, the same trie supports incremental updates; with a static dictionary, sorting roots and using a hash set of all prefixes is an alternative but heavier on memory.

9.4 Design Search Autocomplete System (LC 642)

Problem. Design a system seeded with historical sentences and their times (counts). `input(c)` is fed one character at a time; for each non-`#` character it returns the top 3 historical sentences whose prefix matches what's been typed this session, ranked by descending frequency then ascending ASCII. The special character `#` ends the current sentence, records/increments it, and resets the typed prefix (returns []).

Optimal — trie with per-node sentence counts · Time $O(p + m \log m)$ per query, Space $O(\text{total chars} * \text{something})$. p is the current prefix length walked; m is the number of sentences under that prefix node (sorted to pick top 3). Storing sentence- $\rightarrow$ count at each prefix node avoids re-scanning all sentences on every keystroke.

Idea. Build a trie where each node remembers a map of {full sentence $\rightarrow$ count} for every sentence passing through it. On each typed character, descend the trie and rank that node's sentences by $(-\text{count}, \text{sentence})$ to emit the top 3. On `#`, increment the finished sentence's count along its whole path and reset the cursor to the root.

```

1 class TrieNode:
2     def __init__(self):
3         self.children = {}
4         self.counts = {}          # sentence → frequency for everything under this node
5
6 class AutocompleteSystem:
7     def __init__(self, sentences: list[str], times: list[int]):
8         self.root = TrieNode()
9         for s, t in zip(sentences, times):
10             self._add(s, t)
11         self.cur = self.root      # node for the prefix typed so far
12         self.typed = []          # characters typed this session
13         self.dead = False        # True once we fall off the trie
14
15     def _add(self, sentence: str, count: int) → None:
16         node = self.root
17         for ch in sentence:
18             node = node.children.setdefault(ch, TrieNode())
19             node.counts[sentence] = node.counts.get(sentence, 0) + count
20
21     def input(self, c: str) → list[str]:
22         if c == '#':
23             self._add(''.join(self.typed), 1)  # record finished sentence
24             self.cur, self.typed, self.dead = self.root, [], False
25             return []
26         self.typed.append(c)
27         if self.dead or c not in self.cur.children:
28             self.dead = True  # no sentence has this prefix; stop descending
29             return []
30         self.cur = self.cur.children[c]

```

```

31     # rank by frequency desc, then sentence ascending (ASCII)
32     ranked = sorted(self.cur.counts.items(), key=lambda kv: (-kv[1], kv[0]))
33     return [s for s, _ in ranked[:3]]

```

**** Error-prone spots:****

- › Tie-breaking: rank by descending count then ascending lexicographic order; using only count, or descending strings, fails the ordering tests.
- › Forgetting to keep descending state: once a prefix has no matches you must keep returning [] for the rest of the session (the dead flag), even though later characters might coincidentally exist elsewhere in the trie.
- › On '#', you must (a) persist/increment the sentence, (b) reset the prefix, and (c) reset dead; missing any one corrupts later queries.
- › Incrementing counts only at the leaf instead of along the whole path — every prefix node needs the sentence in its counts map for prefix queries to find it.
- › Re-querying a brand-new sentence (first time it's added via '#') must then appear with count 1 in subsequent matching sessions; the path update handles this.

Follow-up. For very large sentence sets, store a small heap or sorted top-k cache at each node instead of re-sorting counts on every keystroke, trading update cost for faster queries.

10 Graphs

Model entities as nodes and relations as edges, then traverse (DFS/BFS) or run a shortest-path algorithm. The grid is an implicit graph where each cell connects to its 4 neighbors.

10.1 Number of Islands (LC 200)

Problem. Given an $m \times n$ grid of '1' (land) and '0' (water), count the number of islands. An island is a maximal group of land cells connected 4-directionally (up/down/left/right). The grid border is surrounded by water. Edge cases: empty grid, all water, all land (one island).

Optimal — DFS/BFS flood fill · Time $O(mn)$, Space $O(mn)$ worst-case recursion/queue. Each cell is visited once. Beats re-scanning.

Idea. Scan every cell; when you hit unvisited land, it is a new island, so increment the counter and flood-fill (sink) the entire connected component to '0' so it is never counted again. The invariant: after processing a cell, all land reachable from it has been marked, so each component contributes exactly one to the count.

```

1 def numIslands(grid: list[list[str]]) → int:
2     if not grid or not grid[0]:
3         return 0
4     rows, cols = len(grid), len(grid[0])
5
6     def sink(r: int, c: int) → None:
7         if r < 0 or r ≥ rows or c < 0 or c ≥ cols or grid[r][c] ≠ '1':
8             return
9         grid[r][c] = '0' # mark visited in-place

```

```

10     sink(r + 1, c); sink(r - 1, c)
11     sink(r, c + 1); sink(r, c - 1)
12
13     count = 0
14     for r in range(rows):
15         for c in range(cols):
16             if grid[r][c] == '1':
17                 count += 1
18                 sink(r, c)
19     return count

```

**** Error-prone spots:****

- › Comparing against integer 1 instead of the string '1' — the grid holds characters.
- › Forgetting bounds/visited checks at the top of sink, causing IndexError or infinite recursion.
- › Counting inside sink instead of once per outer find — double counts.
- › Deep recursion on a huge all-land grid can blow the stack; use an explicit stack/BFS if $m \times n$ is large.
- › Mutating the grid is fine if allowed; if not, keep a separate visited set.

Follow-up. Count islands with diagonal (8-directional) connectivity, or count distinct island shapes (normalize each component's relative coordinates).

10.2 Clone Graph (LC 133)

Problem. Given a reference to a node in a connected undirected graph where Node has an int val and a list[Node] neighbors, return a deep copy of the entire graph. Edge case: input None returns None; a single node with no neighbors returns a copy with an empty list.

Optimal — DFS/BFS with old → new map · Time $O(V+E)$, Space $O(V)$. Visit each node and edge once. The map prevents cycles from causing infinite cloning.

Idea. Maintain a hash map from each original node to its clone. When you first see a node, create its clone and record it before recursing into neighbors; this breaks cycles because a re-encountered node returns its existing clone. Then wire up each clone's neighbor list from the map.

```

1 class Node:
2     def __init__(self, val=0, neighbors=None):
3         self.val = val
4         self.neighbors = neighbors if neighbors is not None else []
5
6 def cloneGraph(node: 'Node') -> 'Node':
7     if node is None:
8         return None
9     clones = {} # original -> clone
10
11     def dfs(orig: 'Node') -> 'Node':
12         if orig in clones:
13             return clones[orig]
14         copy = Node(orig.val)
15         clones[orig] = copy # record BEFORE recursing to break cycles
16         for nei in orig.neighbors:

```

```

17         copy.neighbors.append(dfs(nei))
18         return copy
19
20     return dfs(node)

```

**** Error-prone spots:****

- › Inserting the clone into the map **AFTER** recursing — a cycle then recreates the node endlessly.
- › Returning `None` for empty input is required; forgetting it crashes on the test harness.
- › Using `val` as the map key fails when values are not unique; key by the node object.
- › Sharing the original neighbor list (shallow copy) instead of building a fresh list of clones.

Follow-up. Clone a graph with weighted edges, or a directed graph (same approach; just don't assume symmetry).

10.3 Pacific Atlantic Water Flow (LC 417)

Problem. Given an $m \times n$ matrix `heights`, water can flow from a cell to a 4-directional neighbor of equal or lower height. The Pacific touches the top and left edges; the Atlantic touches the bottom and right edges. Return all cells $[r, c]$ from which water can reach BOTH oceans. Edge cases: 1×1 grid (reaches both), plateaus of equal height.

Optimal — reverse DFS/BFS from oceans · Time $O(mn)$, Space $O(mn)$. Beats $O((m \cdot n)^2)$ per-cell search by reversing the flow.

Idea. Instead of asking "can this cell reach the ocean?", reverse it: start at the ocean-adjacent cells and climb to neighbors with height $\geq$ current. Any cell reachable in this reverse search can send water to that ocean. Run two such floods (Pacific edges, Atlantic edges) and intersect the two reachable sets.

```

1 def pacificAtlantic(heights: list[list[int]]) -> list[list[int]]:
2     if not heights or not heights[0]:
3         return []
4     rows, cols = len(heights), len(heights[0])
5     pacific, atlantic = set(), set()
6
7     def dfs(r: int, c: int, seen: set, prev: int) -> None:
8         if (r < 0 or r >= rows or c < 0 or c >= cols
9             or (r, c) in seen or heights[r][c] < prev):
10            return
11        seen.add((r, c))
12        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
13            dfs(r + dr, c + dc, seen, heights[r][c])
14
15    for c in range(cols):
16        dfs(0, c, pacific, heights[0][c])      # top edge -> Pacific
17        dfs(rows - 1, c, atlantic, heights[rows-1][c]) # bottom -> Atlantic
18    for r in range(rows):
19        dfs(r, 0, pacific, heights[r][0])      # left edge -> Pacific
20        dfs(r, cols - 1, atlantic, heights[r][cols-1]) # right -> Atlantic
21
22    return [[r, c] for r in range(rows) for c in range(cols)
23            if (r, c) in pacific and (r, c) in atlantic]

```

**** Error-prone spots:****

- › Reversing the comparison: climbing UP means the neighbor must be $\geq$ current height, not $\leq$.
- › Seeding only corners instead of the full edge rows/columns.
- › Using one shared `seen` set for both oceans — must be two separate sets.
- › Off-by-one indexing for the bottom row (`rows-1`) and right column (`cols-1`).

Follow-up. Return only the count, or handle diagonal flow.

10.4 Course Schedule (LC 207)

Problem. Given `numCourses` labeled $0 \dots \text{numCourses}-1$ and a list of prerequisite pairs `[a, b]` meaning you must take `b` before `a`, return `True` if you can finish all courses. Equivalent to detecting whether the directed prerequisite graph is acyclic. Edge cases: no prerequisites (always true), self-loop `[a, a]` (false), duplicate edges.

Optimal — Kahn's topological sort (BFS) · Time $O(V+E)$, Space $O(V+E)$. Linear; beats trying permutations.

Idea. Build the graph and an in-degree count. Repeatedly remove nodes with in-degree 0 (no remaining prerequisites), decrementing their neighbors' in-degrees. If every node is eventually removed, a valid ordering exists (no cycle); if some nodes never reach in-degree 0, they sit in a cycle.

```

1 from collections import deque
2
3 def canFinish(numCourses: int, prerequisites: list[list[int]]) → bool:
4     graph = [[] for _ in range(numCourses)]
5     indegree = [0] * numCourses
6     for course, pre in prerequisites:
7         graph[pre].append(course) # edge pre → course
8         indegree[course] += 1
9
10    queue = deque(i for i in range(numCourses) if indegree[i] == 0)
11    taken = 0
12    while queue:
13        node = queue.popleft()
14        taken += 1
15        for nxt in graph[node]:
16            indegree[nxt] -= 1
17            if indegree[nxt] == 0:
18                queue.append(nxt)
19    return taken == numCourses

```

**** Error-prone spots:****

- › Reversing edge direction: `[a, b]` means $b \rightarrow a$; the in-degree belongs to `a` (the course), not `b`.
- › Initializing the queue with all in-degree-0 nodes, not just node 0.
- › Comparing `taken` against `len(prerequisites)` instead of `numCourses`.
- › DFS alternative needs three states (unvisited/visiting/done); a plain visited set misses back-edges.

Follow-up. Course Schedule II (LC 210): return the actual ordering — append nodes to a result

list as they leave the queue.

10.5 Number of Connected Components in an Undirected Graph (LC 323)

Problem. Given n nodes labeled $0 \dots n-1$ and an edge list, return the number of connected components. Edges are undirected; the graph may be disconnected. Edge cases: no edges (n components), all nodes in one component (1).

Optimal – Union-Find (or DFS) · Time $O(n + E \cdot \alpha(n))$, Space $O(n)$. Near-linear with union by rank + path compression.

Idea. Start with n components. Each edge that joins two nodes from different components reduces the count by one; edges within an already-merged component change nothing. Union-Find tracks the merges efficiently; the remaining count is the answer.

```

1 def countComponents(n: int, edges: list[list[int]]) → int:
2     parent = list(range(n))
3     rank = [0] * n
4     count = n
5
6     def find(x: int) → int:
7         while parent[x] ≠ x:
8             parent[x] = parent[parent[x]] # path compression (halving)
9             x = parent[x]
10        return x
11
12    def union(a: int, b: int) → None:
13        nonlocal count
14        ra, rb = find(a), find(b)
15        if ra == rb:
16            return
17        if rank[ra] < rank[rb]:
18            ra, rb = rb, ra
19        parent[rb] = ra
20        if rank[ra] == rank[rb]:
21            rank[ra] += 1
22        count -= 1
23
24    for a, b in edges:
25        union(a, b)
26    return count

```

**** Error-prone spots:****

- Decrementing the count even when both endpoints already share a root — only decrement on a real merge.
- Forgetting path compression / union by rank degrades to $O(n)$ per op on adversarial chains.
- For the DFS approach, count one component per unvisited node started, not per node visited.

Follow-up. Graph Valid Tree (LC 261): a tree iff exactly $n-1$ edges AND one component (no cycle).

10.6 Word Ladder (LC 127)

Problem. Given `beginWord`, `endWord`, and a `wordList`, return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, changing exactly one letter per step, where every intermediate word must be in `wordList`. Return 0 if no sequence exists. All words have equal length. `beginWord` need not be in the list; `endWord` must be. Length counts both endpoints.

Optimal – BFS over the pattern graph · Time $O(N * L^2)$ where L =word length, N =words, Space $O(N*L)$. BFS gives the shortest path in an unweighted graph; the * pattern index avoids $O(N^2)$ pairwise comparisons.

Idea. Treat words as nodes with edges between words differing in one letter. Precompute a map from wildcard patterns like `h*t` to the words matching them, so neighbors are found in $O(L)$ per word instead of scanning all words. BFS layer by layer from `beginWord`; the first time `endWord` is dequeued, its level is the answer.

```

1 from collections import deque, defaultdict
2
3 def ladderLength(beginWord: str, endWord: str, wordList: list[str]) -> int:
4     words = set(wordList)
5     if endWord not in words:
6         return 0
7     L = len(beginWord)
8     patterns = defaultdict(list)
9     for w in words:
10         for i in range(L):
11             patterns[w[:i] + '*' + w[i+1:]].append(w)
12
13     visited = {beginWord}
14     queue = deque([(beginWord, 1)])
15     while queue:
16         word, steps = queue.popleft()
17         if word == endWord:
18             return steps
19         for i in range(L):
20             patt = word[:i] + '*' + word[i+1:]
21             for nei in patterns[patt]:
22                 if nei not in visited:
23                     visited.add(nei)
24                     queue.append((nei, steps + 1))
25     return 0

```

**** Error-prone spots:****

- › Returning 0 when `endWord` is absent from the list — without this you may loop or miscount.
- › Marking a word visited when enqueued (not when dequeued) prevents it being added by multiple parents.
- › Off-by-one on the length: it counts words/levels, so `beginWord` starts at 1, not 0.
- › Rebuilding patterns inside the loop instead of precomputing kills performance.

Follow-up. Bidirectional BFS (search from both ends, swap to the smaller frontier) cuts the explored space roughly to its square root; Word Ladder II (LC 126) returns all shortest paths.

10.7 Network Delay Time (LC 743)

Problem. A network of n nodes labeled $1..n$ with directed weighted edges $\text{times}[i] = [u, v, w]$ (signal travels $u \rightarrow v$ in w time). Starting from node k , return the minimum time for ALL nodes to receive the signal, or -1 if some node is unreachable. Weights are non-negative.

Optimal — Dijkstra with a min-heap · Time $O(E \log V)$, Space $O(V+E)$. Beats Bellman-Ford's $O(V^2E)$ since weights are non-negative.

Idea. Dijkstra computes shortest distances from k to every node. Pop the closest unsettled node, finalize its distance, and relax its outgoing edges. The answer is the maximum finalized distance (the last node to be reached); if any node is never settled, return -1 .

```

1 import heapq
2 from collections import defaultdict
3
4 def networkDelayTime(times: list[list[int]], n: int, k: int) -> int:
5     graph = defaultdict(list)
6     for u, v, w in times:
7         graph[u].append((v, w))
8
9     dist = {}
10    heap = [(0, k)] # (distance, node)
11    while heap:
12        d, node = heapq.heappop(heap)
13        if node in dist:
14            continue # already finalized with a smaller distance
15        dist[node] = d
16        for nei, w in graph[node]:
17            if nei not in dist:
18                heapq.heappush(heap, (d + w, nei))
19    return max(dist.values()) if len(dist) == n else -1

```

**** Error-prone spots:****

- › Nodes are 1-indexed; checking $\text{len}(\text{dist}) == n$ (not $n+1$ or $n-1$) for full coverage.
- › Skipping the "already finalized" guard lets stale, larger entries overwrite or do redundant work.
- › Returning the sum of distances instead of the max — the network is done only when the slowest node receives it.
- › Forgetting -1 when some node is unreachable.

Follow-up. With negative edges (no negative cycles), use Bellman-Ford or SPFA instead.

10.8 Cheapest Flights Within K Stops (LC 787)

Problem. Given n cities and flights $[\text{from}, \text{to}, \text{price}]$, find the cheapest price from src to dst using AT MOST k stops (i.e., at most $k+1$ edges). Return -1 if no such route exists. Prices are non-negative; the graph may contain cycles.

Optimal — Bellman-Ford, $k+1$ relaxation rounds · Time $O(k \cdot E)$, Space $O(V)$. The stop limit makes plain Dijkstra incorrect (cheapest path may use more stops); bounded relaxations naturally cap edge count.

Idea. Relax all edges exactly $k+1$ times. After round i , $\text{dist}[v]$ holds the cheapest cost reaching v using at most i edges. Crucially, each round must read from a frozen snapshot of the previous

round's distances so a single round cannot chain multiple edges; copying `dist` enforces the "at most k edges" invariant.

```

1 def findCheapestPrice(n: int, flights: list[list[int]], src: int,
2                       dst: int, k: int) → int:
3     INF = float('inf')
4     dist = [INF] * n
5     dist[src] = 0
6     for _ in range(k + 1):          # at most k stops == k+1 edges
7         snapshot = dist[:]          # freeze: prevents using >1 new edge per round
8         for u, v, w in flights:
9             if snapshot[u] ≠ INF and snapshot[u] + w < dist[v]:
10                 dist[v] = snapshot[u] + w
11     return dist[dst] if dist[dst] ≠ INF else -1

```

**** Error-prone spots:****

- Not snapshotting `dist` each round: edges relaxed in the same pass chain together, exceeding the stop limit and returning too-cheap prices.
- Running k rounds instead of $k+1$ (k stops means $k+1$ flights).
- Reading `dist[u]` (live) instead of `snapshot[u]` inside the inner check.
- Using Dijkstra by cost alone is wrong here; if used, the heap state must include the stop count.

Follow-up. A Dijkstra/BFS variant carrying (cost, node, stops) in the heap also works and can be faster when k is large.

11 Union-Find / Disjoint Set

Disjoint-set maintains a partition of elements into components, supporting near- $O(1)$ union and find via union-by-rank and path compression. Reuse the class below throughout this section.

```

1 class UnionFind:
2     def __init__(self, n: int):
3         self.parent = list(range(n))
4         self.rank = [0] * n
5         self.count = n # number of disjoint components
6
7     def find(self, x: int) → int:
8         root = x
9         while self.parent[root] ≠ root:
10             root = self.parent[root]
11         while self.parent[x] ≠ root:      # full path compression
12             self.parent[x], x = root, self.parent[x]
13         return root
14
15     def union(self, a: int, b: int) → bool:
16         ra, rb = self.find(a), self.find(b)
17         if ra == rb:
18             return False                # already connected (cycle/redundant)
19         if self.rank[ra] < self.rank[rb]:

```

```

20         ra, rb = rb, ra
21         self.parent[rb] = ra
22         if self.rank[ra] == self.rank[rb]:
23             self.rank[ra] += 1
24         self.count -= 1
25         return True
26
27     def connected(self, a: int, b: int) → bool:
28         return self.find(a) == self.find(b)

```

11.1 Number of Provinces (LC 547)

Problem. Given an $n \times n$ symmetric matrix `isConnected` where `isConnected[i][j] = 1` means cities i and j are directly connected, return the number of provinces — maximal groups of directly or indirectly connected cities. The diagonal is 1. Edge cases: no connections (n provinces), all connected (1).

Optimal — Union-Find · Time $O(n^2 * \alpha(n))$, Space $O(n)$. Must scan the full matrix; α is effectively constant.

Idea. Each city starts in its own province. Union cities i and j whenever `isConnected[i][j] = 1`. Because the matrix is symmetric, only the upper triangle ($j > i$) needs scanning. The remaining component count is the number of provinces.

```

1 def findCircleNum(isConnected: list[list[int]]) → int:
2     n = len(isConnected)
3     uf = UnionFind(n)
4     for i in range(n):
5         for j in range(i + 1, n): # upper triangle only (symmetric)
6             if isConnected[i][j] == 1:
7                 uf.union(i, j)
8     return uf.count

```

**** Error-prone spots:****

- › Scanning the whole matrix is fine but wasteful; never union i with itself off the diagonal logic.
- › Returning n minus union calls is wrong if some unions are redundant; trust `uf.count` which only decrements on real merges.
- › Misreading the input as an edge list — it is an adjacency matrix.

Follow-up. Equivalent DFS/BFS over the adjacency matrix; same complexity.

11.2 Redundant Connection (LC 684)

Problem. A tree on n nodes labeled $1..n$ had exactly one extra edge added, forming a single cycle (the graph stays connected with n edges). Given the edge list in input order, return the one edge that can be removed so the result is a tree. If multiple answers exist, return the one appearing last in the input.

Optimal – Union-Find · Time $O(n * \alpha(n))$, Space $O(n)$. One pass; beats repeated cycle-search per edge.

Idea. Process edges in order, unioning endpoints. The first edge whose two endpoints are ALREADY in the same component closes a cycle and is therefore redundant. Returning it as soon as found yields the last such edge in input order (there is exactly one extra edge, so the first cycle-closing edge is the answer).

```
def findRedundantConnection(edges: list[list[int]]) → list[int]:
    uf = UnionFind(len(edges) + 1) # nodes are 1..n; index 0 unused
    for a, b in edges:
        if not uf.union(a, b):      # union returns False when already connected
            return [a, b]
    return []
```

**** Error-prone spots:****

- › Nodes are 1-indexed; size the structure $n+1$ so index n is valid (and ignore index 0).
- › Returning the edge whose union SUCCEEDS instead of the one that fails the union (already connected).
- › Over-thinking "last in input": scanning forward and returning the first cycle edge already satisfies it.

Follow-up. Redundant Connection II (LC 685): directed version where you must also handle a node with two parents; combine in-degree checks with Union-Find.

11.3 Accounts Merge (LC 721)

Problem. Given accounts where `accounts[i] = [name, email1, email2, ...]`, merge accounts that share any common email (they belong to the same person). Two accounts with the same name but no shared email are different people. Return the merged accounts, each as `[name, sorted_emails...]`. The output order of accounts is flexible.

Optimal – Union-Find over emails · Time $O(NK\alpha + \text{sorting})$, Space $O(N*K)$ where N =accounts, K =avg emails. Sorting emails dominates: $O(\text{total} * \log \text{total})$.

Idea. Treat every email as a node. Within one account, union all its emails to the first email, linking that account's emails into one component. Across accounts, a shared email lives in only one component, so its presence merges those accounts transitively. Group emails by their component root, then attach the owner's name (tracked per email).

```
from collections import defaultdict

def accountsMerge(accounts: list[list[str]]) → list[list[str]]:
    emails = []
```

```

5     email_id = {}
6     email_name = {}
7     for acct in accounts:
8         name = acct[0]
9         for e in acct[1:]:
10            if e not in email_id:
11                email_id[e] = len(emails)
12                emails.append(e)
13                email_name[e] = name
14
15     uf = UnionFind(len(emails))
16     for acct in accounts:
17         first = email_id[acct[1]]
18         for e in acct[2:]:
19             uf.union(first, email_id[e])
20
21     groups = defaultdict(list)
22     for e, idx in email_id.items():
23         groups[uf.find(idx)].append(e)
24
25     return [[email_name[grp[0]]] + sorted(grp) for grp in groups.values()]

```

**** Error-prone spots:****

- An account may legitimately have a single email; `acct[2:]` is empty then, which is fine — don't index `acct[1]` blindly without it existing (every account has a name + at least one email per constraints).
- Using the name as a merge key — names are NOT unique; only emails identify a person.
- Forgetting to sort the emails in each output group.
- Mapping email → name must use the last/any owner; all accounts sharing an email have the same name by problem guarantee.

Follow-up. Return the number of distinct people instead of the merged lists — just count groups.

11.4 Satisfiability of Equality Equations (LC 990)

Problem. Given an array of strings of the form `"a=b"` or `"a≠b"` (single lowercase variable on each side), return `True` if you can assign integers to variables so that all equations hold. Edge cases: `"a≠a"` is immediately unsatisfiable; `"a=a"` is always fine.

Optimal — Union-Find · Time $O(N \cdot \alpha(26))$, Space $O(1)$ (fixed 26 letters). Two passes over the equations.

Idea. Equality is transitive, so first union all variables joined by `=` into components. Then check every `≠` equation: if its two sides are in the SAME component they are forced equal, contradicting the inequality, so return `False`. Processing all equalities before any inequality is essential because of transitivity.

```

1 def equationsPossible(equations: List[str]) → bool:
2     uf = UnionFind(26)
3     idx = lambda ch: ord(ch) - ord('a')
4     for eq in equations:

```

```

5         if eq[1] == '=':                                # "x=y"
6             uf.union(idx(eq[0]), idx(eq[3]))
7     for eq in equations:
8         if eq[1] == '!':                                # "x≠y"
9             if uf.connected(idx(eq[0]), idx(eq[3])):
10                return False
11     return True

```

**** Error-prone spots:****

- Processing equalities and inequalities in a single pass — a later = can retroactively violate an earlier-checked ≠. Do equalities first.
- Indexing the operator: characters are at positions 0,1,2,3 (a,=,!,=,b); the right variable is eq[3], not eq[2].
- Not handling "a≠a" — it is caught automatically since a is connected to itself.

Follow-up. Generalize to a system with < / > constraints — that needs a difference/inequality graph, not plain Union-Find.

12 LRU / LFU Cache

Constant-time caches combine a hash map (for $O(1)$ key lookup) with an ordering structure (doubly linked list or per-frequency buckets) to find and evict the right entry in $O(1)$.

12.1 LRU Cache (LC 146)

Problem. Design a cache with capacity c supporting `get(key)` and `put(key, value)`, each in $O(1)$. `get` returns the value or -1 if absent and marks the key most-recently-used. `put` inserts/updates and marks most-recently-used; if over capacity, evict the LEAST-recently-used key. Capacity ≥ 1 .

Optimal — hashmap + doubly linked list · Time $O(1)$ per op, Space $O(C)$. The DLL gives $O(1)$ move-to-front and tail eviction; dict gives $O(1)$ lookup.

Idea. A doubly linked list orders nodes from most-recently-used (head) to least (tail) with sentinel head/tail nodes to avoid null checks. The dict maps key $\rightarrow$ node for $O(1)$ access. Every access unlinks the node and re-inserts it after head; eviction removes the node before tail. (Python's `OrderedDict` does this in a few lines, but the from-scratch DLL is shown as requested.)

```

1 class _Node:
2     __slots__ = ('key', 'val', 'prev', 'next')
3     def __init__(self, key=0, val=0):
4         self.key, self.val = key, val
5         self.prev = self.next = None
6
7 class LRUCache:
8     def __init__(self, capacity: int):
9         self.cap = capacity
10        self.map = {}                # key → _Node
11        self.head = _Node()         # sentinel MRU side

```



```

12     self.tail = _Node()                # sentinel LRU side
13     self.head.next = self.tail
14     self.tail.prev = self.head
15
16     def _remove(self, node: _Node) → None:
17         node.prev.next = node.next
18         node.next.prev = node.prev
19
20     def _add_front(self, node: _Node) → None:
21         node.prev = self.head
22         node.next = self.head.next
23         self.head.next.prev = node
24         self.head.next = node
25
26     def get(self, key: int) → int:
27         if key not in self.map:
28             return -1
29         node = self.map[key]
30         self._remove(node)
31         self._add_front(node)           # mark most-recently-used
32         return node.val
33
34     def put(self, key: int, value: int) → None:
35         if key in self.map:
36             self._remove(self.map[key])
37         node = _Node(key, value)
38         self.map[key] = node
39         self._add_front(node)
40         if len(self.map) > self.cap:
41             lru = self.tail.prev        # node just before tail sentinel
42             self._remove(lru)
43             del self.map[lru.key]       # need key → hence store key in node
44         return

```

**** Error-prone spots:****

- Forgetting to store the key inside the node — you need it to delete from the map during eviction.
- Not re-inserting on get: an accessed key must become most-recently-used.
- On put of an existing key, failing to remove the old node first leaves a stale duplicate in the DLL.
- Mixing up which sentinel is MRU vs LRU; evict from `tail.prev`, insert at `head.next`.
- Pointer update order in `_add_front`: set the new node's links before rewiring `head.next`.

Follow-up. `OrderedDict` one-liner: `move_to_end(key)` on access and `popitem(last=False)` to evict — same $O(1)$ behavior with far less code.

12.2 LFU Cache (LC 460)

Problem. Design a cache with capacity c where get/put are $O(1)$. Evict the LEAST-FREQUENTLY-used key on overflow; break ties by evicting the LEAST-RECENTLY-used among the lowest frequency. Both get and put (on an existing key) increment that key's use frequency. Capacity may be 0 (every put is a no-op).

Optimal — hashmap + frequency buckets of OrderedDicts · Time $O(1)$ per op, Space $O(C)$. Tracking min_freq and per-frequency insertion-ordered buckets gives $O(1)$ eviction.

Idea. Keep val and freq per key. For each frequency value, hold an `OrderedDict` of keys at that frequency in LRU order (oldest first). On access, move the key from bucket f to bucket $f+1$; if bucket f empties and $f = \text{min_freq}$, increment min_freq . Eviction pops the oldest key from the min_freq bucket. A new key enters at frequency 1, resetting min_freq to 1.

```

1 from collections import defaultdict, OrderedDict
2
3 class LFUCache:
4     def __init__(self, capacity: int):
5         self.cap = capacity
6         self.val = {}           # key → value
7         self.freq = {}         # key → frequency
8         self.buckets = defaultdict(OrderedDict) # freq → {key: None} in LRU order
9         self.min_freq = 0
10
11     def _bump(self, key: int) → None:
12         f = self.freq[key]
13         del self.buckets[f][key]
14         if not self.buckets[f]:
15             del self.buckets[f]
16             if self.min_freq == f:
17                 self.min_freq += 1
18         self.freq[key] = f + 1
19         self.buckets[f + 1][key] = None
20
21     def get(self, key: int) → int:
22         if key not in self.val:
23             return -1
24         self._bump(key)
25         return self.val[key]
26
27     def put(self, key: int, value: int) → None:
28         if self.cap == 0:
29             return
30         if key in self.val:
31             self.val[key] = value
32             self._bump(key)
33             return
34         if len(self.val) ≥ self.cap:
35             evict, _ = self.buckets[self.min_freq].popitem(last=False) # oldest
36             del self.val[evict]
37             del self.freq[evict]
38         self.val[key] = value
39         self.freq[key] = 1
40         self.buckets[1][key] = None
41         self.min_freq = 1 # new key has the lowest frequency

```

**** Error-prone spots:****

- › Resetting `min_freq = 1` on every NEW insertion is mandatory; forgetting it evicts the wrong key next time.
- › Updating `min_freq` only when the emptied bucket equals the current `min_freq` — not on every bump.
- › Handling `capacity = 0` up front, or `popitem` will fail on an empty bucket.
- › `popitem(last=False)` evicts the OLDEST (LRU) within the min-freq bucket; `last=True` would be wrong.
- › On put of an existing key, you must both update the value AND bump frequency.

Follow-up. A from-scratch version uses a doubly linked list of frequency nodes, each holding its own DLL of entries — equivalent $O(1)$ but more code than the `OrderedDict` buckets.

13 Segment Tree & Fenwick Tree (BIT)

A Fenwick tree (Binary Indexed Tree) maintains prefix aggregates with $O(\log n)$ point update and prefix query using the low-bit trick; segment trees generalize to arbitrary range queries. Both turn "update an element and re-query a range" from $O(n)$ into $O(\log n)$.

13.1 Range Sum Query --- Mutable (LC 307)

Problem. Given an integer array `nums`, support `update(index, val)` (set `nums[index] = val`) and `sumRange(left, right)` (inclusive sum). Many interleaved operations; naive prefix sums make updates $O(n)$. Edge cases: single-element ranges, updating to the same value.

Optimal — Fenwick tree (BIT) · Time $O(\log n)$ per op, $O(n)$ build, Space $O(n)$. Beats $O(n)$ update (prefix array) and $O(n)$ query (raw array).

Idea. A BIT stores partial sums indexed so that `tree[i]` covers a range of length `i & -i` ending at `i`. A prefix sum walks downward by stripping the low bit; an update walks upward adding the low bit. update is implemented as a delta: add (`val - old`) so we adjust rather than overwrite. `sumRange(l, r) = prefix(r) - prefix(l-1)`.

```

1 class NumArray:
2     def __init__(self, nums: list[int]):
3         self.n = len(nums)
4         self.nums = [0] * self.n
5         self.tree = [0] * (self.n + 1)    # 1-indexed BIT
6         for i, v in enumerate(nums):
7             self.update(i, v)
8
9     def _add(self, i: int, delta: int) → None:
10        i += 1                                # to 1-indexed
11        while i ≤ self.n:
12            self.tree[i] += delta
13            i += i & -i                        # move to next responsible node
14
15    def _prefix(self, i: int) → int:          # sum of nums[0..i], i is 0-indexed
16        i += 1
17        s = 0
18        while i > 0:
19            s += self.tree[i]
20            i -= i & -i                        # strip low bit

```

```

21         return s
22
23     def update(self, index: int, val: int) → None:
24         self._add(index, val - self.nums[index]) # apply delta
25         self.nums[index] = val
26
27     def sumRange(self, left: int, right: int) → int:
28         return self._prefix(right) - (self._prefix(left - 1) if left > 0 else 0)

```

**** Error-prone spots:****

- › Off-by-one between 0-indexed API and 1-indexed BIT; convert at the boundary ($i \pm 1$).
- › update must apply the DELTA $val - old$, not the absolute value; forgetting to refresh $self.nums[index]$ corrupts future deltas.
- › $sumRange(0, r)$: guard $left - 1 = -1$ so you don't call $_prefix(-1)$.
- › Sizing tree as $n+1$, not n , because it is 1-indexed.

Follow-up. A segment tree handles min/max/gcd ranges where a BIT cannot (no inverse operation); for range-update + range-query use a BIT of differences or a lazy segment tree.

13.2 Count of Smaller Numbers After Self (LC 315)

Problem. Given an integer array `nums`, return an array `counts` where `counts[i]` is the number of elements to the RIGHT of i that are strictly smaller than `nums[i]`. Values may be negative and duplicated. Edge cases: empty array, all equal (all zeros), strictly increasing (all zeros), strictly decreasing.

Optimal — BIT over compressed ranks · Time $O(n \log n)$, Space $O(n)$. Beats the $O(n^2)$ brute force of counting per element.

Idea. Process from right to left. Maintain a BIT indexed by the value's rank (after coordinate compression of the distinct sorted values). For each element, query how many already-seen elements have a strictly smaller rank (a prefix sum up to $rank-1$), then insert the current element's rank. Compression bounds the BIT size to the number of distinct values.

```

1 import bisect
2
3 class _BIT:
4     def __init__(self, n: int):
5         self.tree = [0] * (n + 1)
6     def add(self, i: int) → None:          # i is 1-indexed rank
7         while i < len(self.tree):
8             self.tree[i] += 1
9             i += i & -i
10    def query(self, i: int) → int:         # count of ranks in [1..i]
11        s = 0
12        while i > 0:
13            s += self.tree[i]
14            i -= i & -i
15        return s
16
17 def countSmaller(nums: list[int]) → list[int]:
18     sorted_vals = sorted(set(nums))
19     rank = {v: i + 1 for i, v in enumerate(sorted_vals)} # 1-indexed ranks

```

```

20     bit = _BIT(len(sorted_vals))
21     res = [0] * len(nums)
22     for i in range(len(nums) - 1, -1, -1):
23         r = rank[nums[i]]
24         res[i] = bit.query(r - 1)          # strictly smaller → ranks < r
25         bit.add(r)
26     return res

```

**** Error-prone spots:****

- Querying r instead of $r - 1$ counts equal values too; "strictly smaller" needs $r - 1$.
- Iterating left-to-right counts the wrong side; must go right-to-left so the BIT holds only elements to the right.
- Forgetting coordinate compression makes the BIT huge (or impossible) for large/negative values.
- Using 0-indexed ranks breaks the BIT (index 0 is invalid); ranks start at 1.

Follow-up. A modified merge sort that counts inversions during the merge solves it in the same $O(n \log n)$ without compression.

13.3 Reverse Pairs (LC 493)

Problem. Given an integer array `nums`, count pairs (i, j) with $i < j$ and $nums[i] > 2 * nums[j]$. Values can be large and negative, so $2 * nums[j]$ may overflow in fixed-width languages (not in Python). Edge cases: empty/one element (0), the $2*$ factor distinguishes this from plain inversions.

Optimal — modified merge sort · Time $O(n \log n)$, Space $O(n)$. Beats $O(n^2)$; counts during the merge.

Idea. Standard merge sort, but before merging two sorted halves, count cross pairs: for each i in the left half, advance a pointer j in the right half while $left[i] > 2 * right[j]$, accumulating the count. Because both halves are sorted, the pointer j only moves forward across the whole left half, giving $O(n)$ counting per merge level. Then merge as usual to keep the array sorted for parent calls.

```

1 def reversePairs(nums: list[int]) → int:
2     def sort_count(lo: int, hi: int) → int:
3         if hi - lo ≤ 1:
4             return 0
5         mid = (lo + hi) // 2
6         cnt = sort_count(lo, mid) + sort_count(mid, hi)
7         # count cross pairs: left[i] > 2*right[j]
8         j = mid
9         for i in range(lo, mid):
10            while j < hi and nums[i] > 2 * nums[j]:
11                j += 1
12            cnt += j - mid
13            nums[lo:hi] = sorted(nums[lo:hi]) # merge step (sort the window)
14        return cnt
15    return sort_count(0, len(nums))

```

**** Error-prone spots:****

- › The counting MUST happen before re-sorting/merging the window, while both halves are still individually sorted.
- › The cross-pair pointer j should NOT reset for each i — it is monotonic across the left half, preserving $O(n)$.
- › Using $\geq$ instead of $>$ (the condition is strict: $\text{nums}[i] > 2 \cdot \text{nums}[j]$).
- › A BIT alternative needs compression of both $\text{nums}[j]$ and $2 \cdot \text{nums}[j]$ values together, which is easy to get subtly wrong.

Follow-up. A BIT over compressed values of nums and $2 \cdot \text{nums}$ also achieves $O(n \log n)$; count, for each j processed left to right, how many earlier values exceed $2 \cdot \text{nums}[j]$.